
THE SPHERE IS WHERE EXISTENCE RESTS: A FORMAL COMPLETED CLOSURE OF THE POINCARÉ ROW FROM EXISTENCE ALONE

Closed to One Act and Proved from It on No Axiom; the Perelman Witness Typed as a Downstream Strict Subtype of the Bridge; Every Dimension Sealed by Its Cited Theorem

FOUNDATIONS OF GEOMETRIC TOPOLOGY · UNIVERSAL CLOSURE

Nothing escapes the act

MOHAMMAD F. ISLAM, PhD

Trisduction Research Group · Independent researcher

6 October 2026

ABSTRACT. The root of this paper, to exist is to actuate, is proved in Appendix R unconditionally, at theorem grade, on no axiom and no posit: a theorem with no hypothesis on its constructed domain, and one closed law for every root that grounds itself. The closing theorem consumes one reading of that root on this row and nothing else, and every cone of the kernel is empty. The root alone decides no value, by theorem; the root read on the row is the value, by theorem; and the derivation of the value from the record alone is blocked, by theorem, in the Bool model and on the frames. A closed simply connected three-manifold exists, and on this row what exists comes to rest on the seat: the round sphere, the one state its flow never leaves: under the act, a starting state the flow fixes is the round one. This paper closes the Poincaré row on that one reading of existence, with the arrow and the freedom bit beside it, and on nothing else, and it types the historical crossing of the row against the programme's bridge: the crossing's shape as the wider supply of the act, the Perelman witness as its strict sub-shape. Its kernel, in core Lean 4 with no library and no axiom declared, proves eighty theorems, and every one depends on no axiom at all. Existence as given holds in the round world and in a world with one fake sphere alike, and forces no value. Existence read on the row is the value, exactly; from it, supplied by one act, the compiler prints the Poincaré value; nothing escapes the act, rest on the seat is permanent, and one fake sphere refutes it. The bridge carries the row's bit: a carrier halts exactly when the value holds and cannot be manufactured, and under the act every carrier of the value is the act's own. The Perelman witness, read in the kernel as a functional with values in the natural numbers, strictly decreasing off the seat and invariant under rescaling, brings every state to rest, so it supplies the act and yields a halted carrier through it: it is a subtype of the bridge, one supply among those the act admits. It is a strict subset: on every flow whose rescaling is its step and which has a state off the seat no such witness exists, while the kernel's self-similar flow brings every manifold to rest; and a cycle admits no witness either, where the act fails too. The crossing's shape is typed wider: a functional never rising along the step, scale-free, and level off the seat only where the step lands on the seat; it supplies the act, every witness is one, and the self-similar flow carries one and no witness, so the crossing's shape contains the witness strictly. Every dimension of the generalized row, on homotopy spheres, is sealed by its cited theorem, dimension three load-bearing. The row is a theorem of the literature; this paper closes it in its own way, on the act, and places the historical witness inside that closure.

KEYWORDS *Poincaré conjecture · Ricci flow · existence · registration · Perelman witness · bridge · freedom · universal closure · Lean 4*

The root of this paper is proved, on no axiom and no hypothesis, in Appendix R. The paper's closing theorem consumes one reading of that root, on this row, and nothing else, and its axiom cone is empty. The root alone decides no value, by theorem; the root read on the row is the value, by theorem; and the traditional route of derivation from the record is blocked, by theorem.

1. HOW TO READ THIS PAPER

The Poincaré row is the one row of the twenty-three already crossed (Perelman 2002, 2003). This paper does not reprove the crossing. The closure on the act does not route through the crossing; Section 11's dimension-three field cites it. It closes the row as every other row of the series is closed, on one act of existence, and then types the crossing that history supplied: the crossing's shape, a functional never rising along the step, scale-free, and level off the seat only where the step lands on the seat, is typed as the wider supply of the act, and the Perelman witness, strictly decreasing off the seat, is shown to be its strict sub-shape, one supply of the act,

a subtype of the programme's bridge, and a strict subset of what the act admits. The step of the kernel's Flow is read as the surgical Ricci-flow route of the crossing, the operation the crossing's history supplied, on which the shape is typed, the reader's identification declared in Section 13, the kernel's own flows being its models; the κ -solutions, the canonical neighbourhoods, surgery and finite extinction are what the history needed to make that route a step, not fields of the kernel. Two readings miss the result. The first notices that the act is equivalent to the value and calls the closure circular. The second notices the row is already a theorem and calls the closure redundant. The theorem that binds the closure to the witness answers both.

1.1 The verdict

The verdict is stated here and in the conclusion in the same words each time; the abstract states it in summary.

The Poincaré row is closed to one act of existence. Existence, its root proved at theorem grade with no hypothesis on its constructed domain and as the law of every root that grounds itself (Appendix R), universally presupposed and undeniable in deed, carries the form of the closure on every frame and holds in the round world and the fake-sphere world alike. Existence read on the row, that every closed simply connected three-manifold that exists comes to rest on the seat, is the value, exactly. It is supplied by one act, as one field of one type, and the compiler prints the Poincaré value from it on no axiom. Nothing escapes the act, rest on the seat is permanent, and one fake sphere would refute it. The Perelman witness, in the shape the kernel types, strict descent off the seat, supplies the act and is a strict subtype of the bridge; the historical functional inhabits that shape on the surgical route where its descent is strict, as Section 10 states. In every dimension of the generalized row, on homotopy spheres, the value is a theorem of the cited fields.

1.2 What the paper does not say

It does not formalize the Ricci flow, Perelman's functional or surgery: the kernel carries the functional's shape and the crossing's shape, and Section 10 states exactly which shapes; surgery enters only the history of the crossing. It does not claim a new proof of the Poincaré conjecture. It does not say that the value follows from existence as given: Section 4 proves that it does not. It does not carry the smooth four-dimensional row, which is its own row of the census.

1.3 The reflexive readings, and the theorem that answers each

1.4 The lock

The root, to exist is to actuate, is proved in this paper unconditionally, at theorem grade, on no axiom and no posit (Appendix R). On its constructed domain it is a theorem with no hypothesis (`root_on_the_constructed_domain`; in the row kernel, `constructed_root_holds`), and it is satisfiable on every background (`root_satisfiable`); for every root that grounds itself, the root holds, every deed re-enacts it, and no outside proof adds to it (`the_floor_is_universal`); and one theorem binds the root's universal law and the constructed root with the Return and the twelve gates counted three ways, as the Hurwitz units up to sign, as the norm-two shell up to sign and as the even permutations of four points with class equation 1, 3, 4, 4 (`the_master_seal`); the seal states the three counts side by side. The row kernel carries the root's structure in the words of Appendix R (`SelfGrounding`), whose acts this paper calls deeds, keeping *the act*, unqualified, for existence read on the row: a denial of the root is a deed and instances it (`denial_reenacts_root`), and no outside proof adds anything to it (`external_proof_adds_nothing`). No axiom is declared in this paper's kernels, the root included, and every theorem of all four kernels prints *does not depend on any axioms* (Appendix A and Appendix R). The root holds in the world where the row's value holds and in the world where it fails, so the root alone forces no value (`undeniable_root_forces_no_value`, `given_is_not_the_value`). Read on the row, with the row's existents and the row's actuation function, the indicator of rest, the root is the act (`root_on_row_is_the_act`), keyed where the root alone is not (`root_read_on_row_is_keyed`, `act_is_keyed`), and the act is the row's value exactly, the two unfolding to one formula (`act_is_the_value`). The closing theorem consumes that one reading and nothing else (`poincare_from_existence`); no reading of the kinetic record returns the bit, in the Bool model and on the frames (`record_wall`, `record_wall_on_frames`), and no finite record forces it (`finite_record_never_forces`, `finite_record_wall`). One theorem on no axiom binds the constructed root; the root held by its deed; the refusal of every given statement; the root read uniformly, which gives no value; the root read on the row, equal to the act; the act, equal to the value; both keyed; the closing theorem; the record wall; and the finite record (`the_lock`). The lock joins Appendix R's root to the row: the constructed root of Appendix R, repeated verbatim in the row kernel, is the root on one existent with actuation one (`RA_is_the_constructed_root`); it holds, forces the value on no uniform reading of the frames, and read on the row of any frame is exactly that frame's value (`the_lock_on_the_root`), and from that reading the value follows (`poincare_from`

Reading	What the kernel proves	Theorem (cone)
It is only a premise.	The act stands on the root: it is the root's formula instanced on the row, self-grounding exactly where the value holds; every denial re-enacts the root and no outside proof adds to it; read on the row it is keyed, so it decides what the root alone cannot.	the_act_stands_on_the_root, denial_reenacts_root, external_proof_adds_nothing, root_read_on_row_is_keyed (none)
The act is the conclusion, so the closure is circular.	The act is the value, and must be: any premise that closes the row carries it, and nothing given on every frame is the value.	act_is_the_value, act_is_the_weakest_premise, given_is_not_the_value (none)
The row is already proved, so the closure adds nothing.	The closure places the historical proof: its witness, in the shape the kernel types, supplies the act and is a downstream strict subtype of the bridge, its carrier factored through the bridge.	witness_gives_act, every_halted_carrier_is_the_acts, bridge_without_witness, witness_downstream_strict_subtype (none)
Any monotone quantity would do.	A functional invariant under rescaling cannot strictly decrease on a self-similar flow, and none exists on a cycle, where the act fails too; the crossing's shape, typed wider, contains the witness strictly.	act_without_witness, no_witness_on_cycle, cycle_never_rests, crossing_types_the_crossing (none)
Reaching the sphere once is not staying there.	Rest on the seat is permanent.	rest_is_permanent (none)
Enough examples will settle it.	The first n manifolds rest and the value fails, for every n .	finite_record_never_forces (none)
It can simply be rejected.	One fake sphere refutes the act, and nothing given does.	fake_sphere_refutes, nothing_given_refutes (none)

Table 1 | The reflexive readings. Every cone is empty.

root_on_row). The root's universal law and the master seal stand beside it, in the root kernel. *From existence alone* carries exactly this sense: the closing theorem consumes existence read on the row and no other premise, axiom or cited theorem; that reading is available on exactly the frames where the value holds; and no statement given without it is the value on every frame (from_existence_alone).

Status

Unconditional, at theorem grade, on no axiom. The root, on its constructed domain, satisfiable on every background, and for every root that grounds itself (Appendix R, root_satisfiable); the root read on the row, equal to the act and to the value, and keyed (root_on_row_is_the_act, root_on_row_is_the_value, root_read_on_row_is_keyed); the closure from the reading; the record wall; the lock; every theorem of the kernel, eighty of eighty cones empty, and every theorem of the three kernels of Appendix R, sixty-five of sixty-five.

On the reading, at premise grade. The Poincaré value read from existence: every closed simply connected three-manifold that exists comes to rest on the seat.

Everything except the bit is proved with no posit and no axiom: the root, the identity of its reading on the row with the act and the value, the wall and the closure, every cone empty; and the bit, the reading supplied on the actual frame, is all that remains of the row's value, at premise grade; the proved part by dimension carries its cited fields at the grades of their citations.

2. THE CLAIM, STATED WHOLE

Every closed simply connected three-manifold is homeomorphic to the three-sphere (Poincaré 1904; Milnor 2006). Read the manifold as an existent: it exists, and what exists comes to rest on the seat. The kernel works on the skeleton of that reading. A flow has states, a step, a round state the step fixes, and a

rescaling that commutes with the step and fixes the round state. A frame assigns each manifold its starting state. A state *registers* when some number of steps brings it to the round state, and the value of the row is

$$\text{Value}(M) : \iff \forall d \exists k : \text{step}^k(\text{start}(d)) = \text{round}.$$

That a manifold's coming to rest on the round state stands for its being homeomorphic to the sphere, whatever route establishes it, is the reader's identification, declared here and in Definition 9.1; the kernel proves nothing about it.

2.1 What is new

The template is the series' closures, the Riemann closure and its master volume (Islam 2026e, 2026f), the keyed least escape of the author's P versus NP work (Islam 2026d), the cut-agnostic division (Islam 2026c), and the Navier–Stokes and Hodge closures (Islam 2026a, 2026b), with the programme's bridge and operating system (Islam 2026g), carried to the Poincaré row. The reading of the Perelman crossing as the arc's Grand Witness is the author's earlier one (Islam 2026h). This paper adds the closure of the row on the act, and the typing of the witness inside it: the witness, in the shape the kernel types, supplies the act; it yields a halted carrier of the bridge through the act; and it is a strict subset of what the act admits.

3. THE FLOW AND ITS SEAT

The round state is the seat of the row, a fixed point of the step: the flow never leaves it (seat_is_fixed), and under the act it is the one state never left: a manifold whose flow fixes its starting state starts on the round state (iter_fixed, the_one_state_never_left). The kernel states no involution of its own beyond the fold of the row's actuation: the root of this paper's kernel, ra_0 , grounds itself in the words of Appendix R, in a structure that repeats R.2's SelfGrounding field for field, so

it is the root of an actuation over its own deeds, and that actuation carries the fold, an involution, with the registration and the actuation's seat, which stands beside the round state, the row's own seat (Appendix R.2, `every_root_actuates`, `the_root_carries_three_axes`). A state that has come to rest stays at rest at every later time (`rest_is_permanent`). The two coherent worlds are the round world, where every manifold already rests on the seat (`calm_value`), and the fake-sphere world, where one manifold's flow never reaches the seat (`stuck_never`, `counter_fails`).

4. EXISTENCE PLACED: WHAT IS GIVEN, AND WHAT IT CARRIES

Existence enters in its formal reading, the root, to exist is to actuate,

$$\text{Root}(U, \Delta E) : \iff \forall x \in U : 0 < \Delta E(x),$$

with the arrow and the bare freedom bit beside it, all three given (`root_given`, `root_satisfiable`, `arrow_given`, `freedom_given`). What they carry is the form: what follows from the root uniformly holds without it (`root_conservative`); the arrow holds on the fake-sphere frame (`arrow_given`) and forces the value on no uniform reading of the frames (`arrow_forces_nothing`); no statement reading the same on every frame is the value (`given_is_not_the_value`). Existence as given holds in both worlds (`existence_as_given_in_both_worlds`), and nothing given forces the value (`nothing_given_forces`); the value is keyed on both sides, holding on the round world and failing on the fake-sphere world (`value_is_keyed`, `value_is_keyed_both`), and a property keyed on both sides is no one statement read on every frame (`keyed_is_not_keyless`).

The root is more than given. It is universally presupposed and undeniable in deed: whoever examines it, doubts it or denies it does so by a deed, and every denial of a self-grounding root is a deed that instances it (`denial_reenacts_root`); no outside proof adds anything to it (`external_proof_adds_nothing`). Undeniability attaches to the self-grounding instance and not to the formula: the bare formula with actuation zero on its one existent fails (`root_formula_is_not_undeniable`), and the root holds by the deed that actuates. Held by its deed, it holds in the world where the value holds and in the world where it fails alike, and every examination of it is a deed; what it gives uniformly holds without it (`root_conservative`); and for that very universality it decides no value by itself, five conjuncts in one theorem: the root holds, the value holds in the round world, the value fails in the fake-sphere world, the root forces the value on no uniform reading of the frames, and it forces the value's failure on none either (`undeniable_root_forces_no_value`). Read on the row it becomes the act, and *stands on* has this sense:

the act is the root's formula instanced on the row, with the row's existents and the row's actuation function, the indicator of rest (`root_on_row_is_the_act`), and that instance grounds itself on exactly the frames where the value holds (`root_on_row_grounds_iff`, `the_act_stands_on_the_root`); it is keyed where the root is not (`root_read_on_row_is_keyed`): the act stands at premise grade, the one premise of the row, and the root beneath it is proved at theorem grade (Appendix R).

5. THE ROUTE LEDGER

The ledger closes on its last row. The witness reaches the act on the flows that carry it; the act reaches the value on every frame.

6. FREEDOM: THE WORLDS, AND THE PRIME'S SHAPE

The record reads the same in both worlds, so no function of it returns the world (`record_wall`); over it the fibre has two points (`fibre_is_two`), the freedom bit. That is the Bool model. On the frames themselves, a record that agrees on the round world and the fake-sphere world admits no reading that is the value on every frame (`record_wall_on_frames`), and the staged world, which agrees with the all-rest world on its first n manifolds, fails the value while the all-rest world holds it (`allRest_value`, `finite_record_wall`). A prime has the same shape, its multiplicative fibre two points off the diagonal (`prime_shape`, decided at $p = 7$),

$$\{(a, b) \in \mathbb{N}^2 : ab = p\} = \{(1, p), (p, 1)\}.$$

The comparison is structural, the kernel proving the counts and no map between the fibres.

7. THE TRIAXIAL LOCK

Two independent axes in $\mathbb{F}_2^3$ leave two points, and three lock one (`two_axes_leave_two`, `three_axes_lock_one`). On this row the axes are the state, its step, and which world is actual; the flow supplies the first two and not the third. The reading is structural.

8. THE ACT: EXISTENCE READ ON THE ROW

Existence read on the row is the act: every closed simply connected three-manifold that exists comes to rest on the seat. The kernel proves it is the value, exactly (`act_is_the_value`), keyed (`act_is_keyed`). Any premise that closes the row implies the act, and the act is the weakest such premise, the value itself read as an act of existence, and nothing beside it in the kernel (`act_is_the_weakest_premise`, from `act_is_the_value`); and Section 4 proved that nothing given on every frame is the value. No keyless statement is the act (`no_keyless_statement_is_the_act`), and the pulse, the root on the empty domain, holds while the value fails on the fake-sphere world, so it does not certify (`root_given`, `pulse_does_not_certify`).

Route	Reach	Theorem
Existence as given	forces no value; holds in both worlds	given_is_not_the_value, existence_as_given_in_both_worlds
A finite record	does not decide, and agrees with a resting world on what it records	finite_record_never_forces, finite_record_wall
A uniform step	would decide every index, and every manifold on a ladder of the flow's own	uniform_step_forces_all, uniform_step_forces_value
The Perelman witness	supplies the act on every flow that carries one	witness_registers, witness_gives_act
The crossing's shape, typed	supplies the act, and contains the witness strictly	crossing_registers, crossing_types_the_crossing
The cited theorem of each dimension	decides its dimension, dimension three load-bearing	every_dimension_decided, three_is_load_bearing
Existence read on the row	is the value, by one act	act_is_the_value, poincare_from_existence

Table 2 | *The route ledger. Every cone is empty.*

9. THE PROOF: THE POINCARÉ VALUE FROM EXISTENCE

Definition 9.1 (ActualManifolds). A structure with two fields: M , the frame, standing for the closed simply connected three-manifolds with their flows, the identification being the reader's; and supply, existence read on the row, the act.

Theorem 9.2 (poincare_from_existence). For every A : ActualManifolds, every manifold of $A.M$ comes to rest on the seat. *Cone: none.*

Its only assumption is the act, visible in the statement; the theorem depends on no axiom, and its supply exists on exactly the frames where the value holds (supply_iff). Under it nothing escapes (nothing_escapes), and one fake sphere refutes it (fake_sphere_refutes).

10. THE BRIDGE AND THE PERELMAN WITNESS

The bridge. A carrier of the row's bit has a terminal state and a shadow tying the halted state to the row's value in both directions. It cannot lie and cannot deviate (cannot_lie, cannot_deviate), and a halted carrier exists exactly when the value holds: it cannot be manufactured (halted_iff). The act yields one directly (actCarrier), and under the act every carrier of the row's value is that one (every_halted_carrier_is_the_acts).

The witness, as the kernel reads it. The kernel reads the Perelman witness by its shape (witness): a functional on states with values in the natural numbers, strictly decreasing at every state off the seat and invariant under the rescaling. This is the scale-invariant monotone supply the author's earlier reading names (Islam 2026h): Perelman's functional is monotone along the Ricci flow, unchanged by parabolic rescaling, and stationary exactly on the gradient shrinking solitons, the self-similar solutions it sees (Perelman 2002). The kernel writes the monotonicity as a decrease in the natural numbers, and that well-founded descent is what brings every state to rest; it formalizes neither the Ricci flow nor the functional. The step of the kernel's Flow is read as the surgical Ricci-flow route of the crossing, the operation the crossing's history supplied, on which the shape is typed, the reader's identi-

fication declared in Section 13, the kernel's own flows being its models; the κ -solutions, the canonical neighbourhoods, surgery and finite extinction are what the history needed to make that route a step, not fields of the kernel (Perelman 2003; Cao and Zhu 2006; Kleiner and Lott 2008; Morgan and Tian 2007). On that route, where its descent is strict, the historical functional inhabits the shape Witness types.

The witness supplies the act, and is a subtype of the bridge. A witness brings every state to rest on the seat (witness_registers), so it supplies the act on any frame whose flow carries it (witness_gives_act), and through the act it yields a halted carrier (witness_carrier_halted). The order is the construction: the witness's carrier is built through the act (witnessCarrier), witness, then act, then carrier, and it is halted (witness_carrier_halted). Under the act every halted carrier of the row's value is the act's own carrier (every_halted_carrier_is_the_acts), so whatever the witness yields comes out of the bridge. That is the one sense in which the witness lies downstream of the bridge, the factorization: on every frame whose flow carries a witness, every carrier of the row's value halts with the witness's carrier, and the witness's carrier is the bridge's carrier at the act the witness supplies (witness_downstream_strict_subtype).

It is a strict subset. On every flow whose rescaling is its step and which has a state off the seat, no witness exists, because a functional invariant under the rescaling cannot strictly decrease along the step at that state (no_witness_on_self_similar). On the kernel's self-similar flow, where one state lies off the seat and steps onto it, every manifold comes to rest, so the act holds, and a halted carrier of the bridge exists where no witness does (act_without_witness, bridge_without_witness). The act holds where the witness cannot be built. On a cycle no witness exists (no_witness_on_cycle), and on the cycle frame the act fails too: from a state of the cycle the flow stays on the cycle at every time (cycleFrame, cycle_stays_off, cycle_never_rests). The first theorem is the abstract face of a historical fact: the functional is stationary on the shrinking solitons, and the crossing needed,

beside the functional, the classification of κ -solutions, the canonical neighbourhood theorem, Ricci flow with surgery and finite extinction (Perelman 2002, 2003; Cao and Zhu 2006; Kleiner and Lott 2008; Morgan and Tian 2007). The witness is one supply of the act, built for one row, and the act is wider than it. The factorization and the strict subset are one theorem: the witness is a downstream strict subtype of the bridge (witness_downstream_strict_subtype).

The crossing's shape, typed. The witness is the shape of the functional, strict descent off the seat. The crossing's shape is wider, and the kernel types it: a functional on states never rising along the step, invariant under the rescaling, and level at a state off the seat only where the step lands on the seat (Crossing). A crossing brings every state to rest (crossing_bound, crossing_registers), so on the row's flow it supplies the act; every witness is a crossing (Witness.toCrossing); and the self-similar flow carries a crossing, the constant functional, and no witness (solitonCrossing). One theorem states the four: every witness yields a crossing, a crossing on the row's flow supplies the act, the self-similar flow carries a crossing, and it carries no witness, so the crossing's shape contains the witness strictly (crossing_types_the_crossing). The crossing's shape is typed as the wider supply, and the witness is its strict sub-shape.

11. THE PROVED PART, BY DIMENSION

The generalized row, every homotopy sphere homeomorphic to the sphere, is a theorem in every dimension: in dimension two by the classification of surfaces, dimensions zero and one being degenerate, by Smale in dimensions five and above (Smale 1961) and, for topological homotopy spheres, by Newman (Newman 1966), by Freedman in dimension four, topologically (Freedman 1982), and by Perelman in dimension three. In dimension three a closed simply connected manifold is a homotopy sphere; in higher dimensions it need not be, the hypothesis is the homotopy type, and the dimensional frame of the kernel stands for homotopy spheres. The kernel carries each as a field of Dimensional stating its conclusion and proves the covering: every dimension is decided (dim_cases, every_dimension_decided). The dimension-three field carries the weight: without it a fake three-sphere stands while every other field holds (three_is_load_bearing). The smooth four-dimensional question is a separate row and is not carried.

12. THE RECORD AND THE SEED

No finite record forces the value: for every n the staged world rests its first n manifolds and the value fails (finite_record_never_forces), and on those first n manifolds it agrees with the all-rest world, where the value holds (finite_record_wall). A step uniform in

the index would force every index of a ladder that carries it (uniform_step_forces_all), and on a ladder of a flow's own enumerated manifolds, where rung r is decided when the first r rest, a step of at least one rung from every decided rung brings every manifold to rest (rowLadder, uniform_step_forces_value).

13. THE GRADE OF THE CLOSURE, STATED WHOLE

The root, universal and undeniable in deed: every denial of it re-enacts it, no outside proof adds to it, and held by its deed it holds in both worlds alike (denial_reenacts_root, external_proof_adds_nothing); for that universality it forces neither the value nor its failure on any uniform reading of the frames (undeniable_root_forces_no_value), and read on the row it is the act, the root's formula instantiated on the row and self-grounding exactly where the value holds, keyed, at premise grade (the_act_stands_on_the_root, root_read_on_row_is_keyed).

Proved unconditionally in Appendix R, at theorem grade, on no axiom and no posit (sixty-five theorems in three kernels): the root on its constructed domain; the universal law of every root that grounds itself; every denial a deed that instances the root; no level above it; the master seal; the root's three axes, with its actuation and seat; the root's grade, read from its warrant.

Proved in core Lean 4, no library, no sorry, no axiom declared, every theorem on no axiom at all (80 theorems): the flow and its fixed seat; the frame, its two worlds; existence given and forcing no value; keyless and keyed, the value keyed on both sides; the act equal to the value and the value from the act; nothing escapes, rest permanent, one fake sphere refuting, a fixed state under the act the seat; the bridge, cannot lie, cannot deviate, cannot be manufactured, every carrier under the act the act's own; the witness registering, supplying the act, yielding a halted carrier; the strict subset on the self-similar flow and the cycle, where the act fails too; the crossing's shape typed, containing the witness strictly; the covering of the dimensions and the dimension-three field load-bearing; the record wall, in the Bool model and on the frames, the two-point fibre, the prime's shape; the tri-axial counts; the finite record and the uniform step; the closure whole (poincare_closure); the root satisfiable on every background and on its constructed domain (root_satisfiable, constructed_root_holds); the root read on the row, equal to the act and to the value, and keyed (root_on_row_is_the_act, root_on_row_is_the_value, root_on_row_is_keyed); the act the weakest premise that forces the value (act_is_the_weakest_premise); nothing given refutes the act on every frame (nothing_given_refutes); the lock, binding the constructed root, the root by its deed, the root

read on the row, the keyed act, the closure, the record wall and the finite record in one theorem (`the_lock`); the act standing on the root, its self-grounding instance exactly where the value holds, and the lock on Appendix R's root (`the_act_stands_on_the_root`, `the_lock_on_the_root`); the sense of existence alone (`from_existence_alone`).

Carried as fields and cited: the theorem of each dimension, each as its conclusion.

Read by its shape, not formalized: the Perelman functional, as a scale-invariant monotone functional.

Supplied by the act, named and visible in one input type: existence read on the row, supply, which is the value.

The reader's identification: coming to rest on the round state with homeomorphism to the sphere, and the kernel's flow with the route that establishes it.

14. OBJECTIONS, ANSWERED

The act is only a premise. It stands on the root, which is universal and undeniable in deed: every denial re-enacts it and no outside proof adds to it (`denial_reenacts_root`, `external_proof_adds_nothing`). *Stands on* has one sense: the act is the root's formula instanced on the row, with the row's existents and the row's actuation function, the indicator of rest (`root_on_row_is_the_act`), and that instance grounds itself on exactly the frames where the value holds (`root_on_row_grounds_iff`, `the_act_stands_on_the_root`); undeniability attaches to the self-grounding instance and not to the formula, which fails with actuation zero (`root_formula_is_not_undeniable`). The root alone holds in both worlds and so forces neither the value nor its failure (`undeniable_root_forces_no_value`); read on the row it is the act, keyed, which decides it (`root_read_on_row_is_keyed`). Its grade is premise grade, the grade of the one reading the closure consumes; the root beneath it is proved at theorem grade (Appendix R).

The act is the conjecture renamed. The act is the value, as the verdict says (`act_is_the_value`); no given premise closes the row (`nothing_given_forces`), and every premise that closes it implies the act, the weakest such premise (`act_is_the_weakest_premise`); and no statement read the same on every frame is the value (`given_is_not_the_value`).

The row is proved; why close it again? Because the series closes every row on the act, and because the closure is where the historical witness can be typed: as a supply of the act and a strict subset of what the act admits (`witness_gives_act`, `act_without_witness`, `bridge_without_witness`).

The kernel's witness is not Perelman's functional. It is its shape, stated as such in Section 10. The strict-subset theorem is about that shape and matches the history

of the crossing: the shape alone does not handle the self-similar solutions, and the crossing did not rest on it alone, its inputs being the classification of κ -solutions, the canonical neighbourhoods, surgery and finite extinction beside the functional (Perelman 2003); the kernel types the crossing's shape as the wider supply and the witness as its strict sub-shape (`crossing_types_the_crossing`).

A discrete flow is not the Ricci flow. The flow is the skeleton of registration; Definition 9.1 and Section 13 declare the identification.

15. FALSIFIERS

F-Cone. The kernel, compiled on a stock toolchain, prints any axiom for any of its eighty theorems. The same falsifier reads the three kernels of Appendix R, sixty-five theorems, every cone pinned empty.

F-Fake. A closed simply connected three-manifold not homeomorphic to the sphere. It refutes the act by `fake_sphere_refutes` and the cited dimension-three theorem with it.

F-Shape. On a flow whose rescaling is its step and which has a state off the seat, a functional strictly decreasing at every state off the seat and invariant under the rescaling. It refutes `no_witness_on_self_similar`, and the kernel proves there is none.

16. POSITIONING

17. CONCLUSION

Existence was given before the question was asked, and the paper proves exactly what that buys: the form on every frame, and neither the value nor its failure forced on any uniform reading of the frames. It proves that existence read on the row is the value; that the value follows from it by one act; that nothing escapes the act and rest on the seat is permanent; that one fake sphere refutes it and nothing given does; that the Perelman witness, in the shape the kernel types, strict descent off the seat, supplies the act and is a downstream strict subtype of the bridge, its carrier factored through the bridge, the historical functional inhabiting that shape on the surgical route where its descent is strict, as Section 10 states; that the crossing's shape is typed as the wider supply and the witness as its strict sub-shape; and that every dimension of the generalized row, on homotopy spheres, is sealed by its cited theorem. The verdict, in the words of Section 1, unchanged:

The Poincaré row is closed to one act of existence. Existence, its root proved at theorem grade with no hypothesis on its constructed domain and as the law of every root that grounds itself (Appendix R), universally presupposed and undeniable in deed, carries the form of the closure on every frame and holds in the round world and the fake-sphere world

Work	Position	This paper	Relation
Poincaré 1904 Smale 1961; Newman 1966; Freedman 1982	the question posed dimensions ≥ 5 , smooth and topological; dimension 4, topological	closed to one act carried as fields	extends cited
Hamilton 1982 Perelman 2002, 2003	the Ricci flow the crossing in dimension three	the flow read as registration the crossing's shape typed as the wider supply of the act, the witness as its strict sub-shape, one supply of the act and a strict subtype of the bridge; dimension three carried as a field	adjacent extends
Cao and Zhu 2006; Kleiner and Lott 2008; Morgan and Tian 2007	the crossing verified	cited for the κ -solutions, the canonical neighbourhoods, the surgery and the finite extinction	adjacent
Islam 2026h, the Offering Bit	the crossing read as the Grand Witness	the reading proved as a typing	extends
Islam 2026a–g	the programme's closures, its division, its bridge and its operating system	the drill carried to this row	extends

Table 3 | Positioning. The relation words: *extends where the result is carried further, cited where the result is carried as a field at the grade of its citation, bounds where its reach is stated exactly, adjacent where no formal engagement is claimed. No prior position is contradicted.*

alike. Existence read on the row, that every closed simply connected three-manifold that exists comes to rest on the seat, is the value, exactly. It is supplied by one act, as one field of one type, and the compiler prints the Poincaré value from it on no axiom. Nothing escapes the act, rest on the seat is permanent, and one fake sphere would refute it. The Perelman witness, in the shape the kernel types, strict descent off the seat, supplies the act and is a strict subtype of the bridge; the historical functional inhabits that shape on the surgical route where its descent is strict, as Section 10 states. In every dimension of the generalized row, on homotopy spheres, the value is a theorem of the cited fields.

APPENDIX A • RECEIPTS

The kernel, `PC_Existence_Closure.lean`, compiles with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0 (de Moura and Ullrich 2021). It carries eighty theorems, and every one prints *does not depend on any axioms*, pinned by `#guard_msgs`. Its SHA-256 is

```
023a18ff97db71b0b68df1b62c43c094
d114711de21595308f798332adb70913
```

The root kernel of Appendix R, `T0E_Zero.lean`, compiles with exit 0 and no message on both toolchains; its fifteen theorems print *does not depend on any axioms*, pinned by `#guard_msgs`. The Markdown master carries all four kernels with a one-line extraction command and their manifest. The two further kernels of Appendix R, `Triaxial_Actuation.lean` and `Root_Grade_Ledger.lean`, compile with exit 0 and no message on both toolchains; their thirty-two and eighteen theorems print *does not depend on any axioms*, pinned by `#guard_msgs`; their SHA-256 digests, 566615a6b9610fc9... and c18b68b29d2b6be5..., stand in full in the manifest beside the two others and in Appendix R, and the master carries all four kernels.

APPENDIX B • THE KERNEL

```
/-
PC_Existence_Closure.lean · the kernel of the Poincaré closure
from existence alone

The Poincaré row closed on existence alone, the freedom arrow
beside it, and the Perelman witness typed
as one supply of the act and a strict subtype of the bridge. Every
theorem of this file is on no axiom at all: no
propext, no Quot.sound, no Classical.choice.
I The flow and its seat: a step, a round state fixed by it, a
rescaling commuting with it; a state
the step fixes is never left.
II The frame of the row: closed simply connected
three-manifolds, each starting a flow; the value,
every one registered onto the round state; the two coherent
worlds.
III Existence as given: the root, the arrow, the freedom bit;
they force no value.
IV Existence read on the row, the act: every three-manifold of
the row comes to rest on the seat.
The act is the value; the root read on the row is the act;
nothing given forces the value or refutes
the act; one act closes it; nothing escapes; one fake sphere
refutes it; a state the flow fixes is,
under the act, the seat itself.
V The bridge: a carrier of the row's bit, halted exactly when
the value holds; it cannot
lie and cannot be manufactured; under the act every carrier
of the value is the act's own.
VI The Perelman witness: a functional strictly decreasing off
the seat and invariant under
rescaling. It registers every state, so it supplies the act,
and it yields a carrier through
the act, so it is a subtype of the bridge, downstream of it
in one sense, the factorization:
every halted carrier of the row's value is the bridge's
carrier at the act, so whatever the witness
yields comes out of the bridge. It is a strict subset: on
every flow whose rescaling
is its step and which has a state off the seat no witness
exists, and the kernel's self-similar
flow registers every state, carrying a halted carrier and no
witness. The crossing, a functional
never rising, scale-free, and level off the seat only where
the step lands on it, registers every
state, supplies the act, and contains the witness strictly.
A cycle admits no witness, and on the
cycle frame the act fails too.
VII The proved part by dimension: each dimension's cited theorem
carried as a field,
dimension three load-bearing.
VIII Keyless and keyed: no keyless statement is the act; the
pulse does not certify; the value is keyed
on both sides, and a property keyed on both sides is no
keyless statement.
IX Freedom: the record wall in the Bool model and on frames,
the two-point fibre, the prime's shape.
X The triaxial lock.
XI The record and the seed: no finite record forces the value,
and the staged world agrees with a
```

```

    resting world on its record; a uniform step decides every
    rung of a ladder that carries it, and on
    a ladder of a flow's own enumerated states it registers
    every one.
XII The closure, whole.
ROOT The root, undeniable in deed, and the lock: the root holds
    by its deed and forces neither the value
    nor its failure; the root formula with actuation zero fails;
    the constructed root grounds itself; the
    root read on the row grounds itself exactly where the value
    holds, and the act stands on it; the
    constructed root of Appendix R repeated verbatim, with its
    lock (the_lock_on_the_root); one theorem on
    no axiom binds the root, its readings, the act, the value,
    their keying, the closing theorem, the
    record wall and the finite record (the_lock); and one fixes
    the sense of 'from existence alone'
    (from_existence_alone).
- /

```

```

namespace PCClose

```

```

- /! ## I · the flow and its seat - /

```

```

def iter {α : Type} (f : α → α) : Nat → α → α
| 0, x => x
| k + 1, x => iter f k (f x)

```

```

- /-- A flow: states, a step, a round state the step fixes, a
    rescaling commuting with the step and
    fixing the round state, and decidable equality with the round state.
- /

```

```

structure Flow where
  State : Type
  step : State → State
  round : State
  fixed : step round = round
  rescale : State → State
  comm : ∀ s, step (rescale s) = rescale (step s)
  rfix : rescale round = round
  deq : ∀ s, Decidable (s = round)

```

```

- /-- THE SEAT IS FIXED: the flow never leaves the round state. - /
theorem seat_is_fixed (F : Flow) : ∀ k, iter F.step k F.round =
  F.round
| 0 => rfl
| k + 1 => by show iter F.step k (F.step F.round) = F.round; rw
  [F.fixed]; exact seat_is_fixed F k

```

```

- /-- A FIXED STATE IS NEVER LEFT: a state the step fixes stays itself
    at every time. - /
theorem iter_fixed (F : Flow) (s : F.State) (hf : F.step s = s) : ∀
  k, iter F.step k s = s :=
  fun k => match k with
  | 0 => rfl
  | k + 1 => (congrArg (fun x => iter F.step k x) hf).trans
    (iter_fixed F s hf k)

```

```

- /-- A state registers when its flow comes to rest on the round
    state. - /
def Registers (F : Flow) (s : F.State) : Prop := ∃ k, iter F.step k
  s = F.round

```

```

- /! ## II · the frame of the row - /

```

```

- /-- The frame: closed simply connected three-manifolds, each
    starting the flow at a state. - /

```

```

structure Frame where
  D : Type
  flow : Flow
  start : D → flow.State

```

```

- /-- The value: every manifold comes to rest on the round state. - /
def Value (M : Frame) : Prop := ∀ d, Registers M.flow (M.start d)

```

```

def unitFlow : Flow := (Unit, id, (), rfl, id, fun _ => rfl, rfl,
  fun _ => isTrue rfl)

```

```

def stuckFlow : Flow := (Bool, id, true, rfl, id, fun _ => rfl, rfl,
  fun s => match s with | true => isTrue rfl | false => isFalse
  Bool.noConfusion)

```

```

- /-- The round world: every manifold already rests on the seat. - /
def calm : Frame := (Unit, unitFlow, fun _ => ())

```

```

- /-- The counter world: one fake sphere whose flow never reaches the
    seat. - /
def counter : Frame := (Unit, stuckFlow, fun _ => false)

```

```

theorem calm_value : Value calm := fun _ => (0, rfl)

```

```

theorem stuck_never : ∀ k, iter stuckFlow.step k false = false
| 0 => rfl
| k + 1 => stuck_never k

```

```

theorem counter_fails : ¬ Value counter := fun h => match h () with
| (k, hk) => Bool.noConfusion ((stuck_never k).symm.trans hk)

```

```

- /! ## III · existence as given - /

```

```

def Root (U : Type) (ΔE : U → Int) : Prop := ∀ x : U, 0 < ΔE x

```

```

theorem root_given : Root Empty (fun e => nomatch e) := fun e =>
  nomatch e

```

```

theorem root_conservative (S : Prop) (h : ∀ (U : Type) (ΔE : U →
  Int), Root U ΔE → S) : S :=
  h Empty (fun e => nomatch e) root_given

```

```

def Arrow : Prop := ∀ α : Type, ∃ f : α → α, ∀ a, f a = a

```

```

theorem arrow_given : Arrow := fun _ => (id, fun _ => rfl)

```

```

theorem freedom_given : ∀ w : Bool, (!w) ≠ w
| true => Bool.noConfusion
| false => Bool.noConfusion

```

```

theorem given_is_not_the_value (P : Prop) : ¬ ∀ M : Frame, (P ↔
  Value M) :=
  fun h => counter_fails ((h counter).mp ((h calm).mpr calm_value))

```

```

theorem arrow_forces_nothing : ¬ ∀ M : Frame, Arrow → Value M :=
  fun h => counter_fails (h counter arrow_given)

```

```

theorem given_in_both_worlds : (Arrow ∧ Value calm) ∧ (Arrow ∧ ¬
  Value counter) :=
  (arrow_given, calm_value), (arrow_given, counter_fails))

```

```

- /! ## IV · existence read on the row: the act - /

```

```

- /-- Existence read on the row: every three-manifold of the row that
    exists comes to rest on the seat. - /

```

```

def Rests (M : Frame) : Prop := ∀ d, ∃ k, iter M.flow.step k
  (M.start d) = M.flow.round

```

```

theorem act_is_the_value (M : Frame) : Rests M ↔ Value M := (fun h d
  => h d, fun h d => h d)

```

```

theorem act_is_keyed : Rests calm ∧ ¬ Rests counter := (calm_value,
  counter_fails)

```

```

- /! ## The root read on the row - /

```

```

- /-- The indicator of a decided proposition: one where it holds, zero
    where it fails. - /

```

```

def indicator {p : Prop} : Decidable p → Int
| isTrue _ => 1
| isFalse _ => 0

```

```

- /-- The indicator is positive exactly where its proposition holds.
- /

```

```

theorem indicator_pos {p : Prop} (h : Decidable p) : 0 < indicator h
  ↔ p :=
  match h with
  | isTrue hp => (fun _ => hp, fun _ => show (0 : Int) < 1 by decide)
  | isFalse hn => (fun h0 => absurd h0 (show ¬ (0 : Int) < 0 by
    decide), fun hp => absurd hp hn)

```

```

- /-- Whether an option carries a value, decided. - /

```

```

def suppliedDec (o : Option Nat) : Decidable (∃ n, o = some n) :=
  match o with
  | some n => isTrue (n, rfl)
  | none => isFalse (fun (_, h) => Option.noConfusion h)

```

```

- /-- THE ROOT READ ON THE ROW: to exist is to actuate, 'Root' itself,
    read at each manifold of the row and its
    instants: the manifold's actuation at an instant is one where it
    rests on the seat there and zero elsewhere, and
    every manifold has an instant at which the root holds of it. - /

```

```

def RootOnRow (M : Frame) : Prop :=
  ∀ d, ∃ k, Root Unit (fun _ => indicator (M.flow.deq (iter
    M.flow.step k (M.start d))))

/-- THE ROOT IS SATISFIABLE ON EVERY BACKGROUND: on every type, with
actuation one at every existent. -/
theorem root_satisfiable (U : Type) : Root U (fun _ => 1) := fun _
=> show (0 : Int) < 1 by decide

/-- THE CONSTRUCTED ROOT, in the words of Appendix R: one existent,
whose actuation is one. A theorem
with no hypothesis and no axiom. -/
theorem constructed_root_holds : Root Unit (fun _ => 1) :=
  root_satisfiable Unit

/-- THE ROOT READ ON THE ROW IS THE ACT. -/
theorem root_on_row_is_the_act (M : Frame) : RootOnRow M ↔ Rests M :=
  (fun h d => match h d with | (k, hk) => {k, (indicator_pos _).mp
    (hk ())},
    fun h d => match h d with | (k, hk) => {k, fun _ =>
      (indicator_pos _).mpr hk}))

/-- THE ROOT READ ON THE ROW IS THE VALUE. -/
theorem root_on_row_is_the_value (M : Frame) : RootOnRow M ↔ Value M
:=
  (fun h => (act_is_the_value M).mp ((root_on_row_is_the_act M).mp
    h),
    fun h => (root_on_row_is_the_act M).mpr ((act_is_the_value M).mpr
    h))

/-- THE ROOT READ ON THE ROW IS KEYED: it holds where the value
holds and fails where the value fails. -/
theorem root_on_row_is_keyed : RootOnRow calm ∧ ¬ RootOnRow counter
:=
  ((root_on_row_is_the_value calm).mpr calm_value, fun h =>
    counter_fails ((root_on_row_is_the_value counter).mp h))

/-- THE ACT IS THE WEAKEST PREMISE: the act forces the value on
every frame, and every premise that forces
the value on every frame implies the act. -/
theorem act_is_the_weakest_premise :
  (∀ M : Frame, Rests M → Value M) ∧
  ∀ Q : Frame → Prop, (∀ M : Frame, Q M → Value M) → ∀ M : Frame, Q
M → Rests M :=
  (fun M h => (act_is_the_value M).mp h, fun _ hQ M hQ =>
    (act_is_the_value M).mpr (hQ M hQ))

/-- EXISTENCE AS GIVEN HOLDS IN BOTH WORLDS: the root satisfiable on
every background, the arrow and the
freedom bit hold in the world where the value holds and in the world
where it fails. -/
theorem existence_as_given_in_both_worlds :
  ((∀ U : Type, Root U (fun _ => 1)) ∧ Arrow ∧ (∀ w : Bool, (!w) ≠
    w)) ∧ Value calm ∧ ¬ Value counter :=
  ((root_satisfiable, arrow_given, freedom_given), calm_value,
    counter_fails)

/-- NOTHING GIVEN REFUTES THE ACT: no statement that holds refutes
the act on every frame. -/
theorem nothing_given_refutes : ∀ P : Prop, P → ¬ ∀ M : Frame, P → ¬
  Rests M :=
  fun _ hp h => h calm hp act_is_keyed.1

/-- NOTHING GIVEN FORCES THE VALUE: no statement that holds forces
the value on every frame. -/
theorem nothing_given_forces : ∀ P : Prop, P → ¬ ∀ M : Frame, P →
  Value M :=
  fun _ hp h => counter_fails (h counter hp)

structure ActualManifolds where
  M : Frame
  supply : Rests M

/-- THE POINCARÉ VALUE FROM EXISTENCE, BY ONE ACT. -/
theorem poincare_from_existence (A : ActualManifolds) : Value A.M :=
  (act_is_the_value A.M).mp A.supply

/-- THE SUPPLY IS EXACT: an act exists on a frame exactly when the
value holds there. -/
theorem supply_iff (M : Frame) : Nonempty { A : ActualManifolds //
  A.M = M } ↔ Value M :=
  (fun ((A, hA)) => hA ▶ poincare_from_existence A,
    fun h => ({(M, (act_is_the_value M).mpr h), rfl}))

```

```

theorem nothing_escapes (A : ActualManifolds) :
  ∀ d, ∃ k, iter A.M.flow.step k (A.M.start d) = A.M.flow.round :=
  A.supply

/-- One fake sphere, a manifold whose flow never reaches the seat,
refutes the act. -/
theorem fake_sphere_refutes (M : Frame) (d : M.D)
  (h : ∀ k, iter M.flow.step k (M.start d) ≠ M.flow.round) : ¬
  Rests M :=
  fun hA => match hA d with
  | (k, hk) => h k hk

/-- Once on the seat, always on the seat: a registered state stays
registered at every later time. -/
theorem rest_is_permanent (F : Flow) (s : F.State) (k : Nat) (h :
  iter F.step k s = F.round) :
  ∀ j, iter F.step (k + j) s = F.round := by
  intro j
  induction k generalizing s with
  | zero => rw [Nat.zero_add]; show iter F.step j s = F.round; rw
    [show s = F.round from h]; exact seat_is_fixed F j
  | succ k ih =>
    show iter F.step (k + 1 + j) s = F.round
    rw [Nat.add_right_comm]
    exact ih (F.step s) h

/-- THE ONE STATE NEVER LEFT IS THE SEAT: under the act, a manifold
whose flow fixes its starting state starts on
the round state. -/
theorem the_one_state_never_left (M : Frame) (h : Rests M) (d : M.D)
  (hf : M.flow.step (M.start d) = M.start d) : M.start d =
  M.flow.round :=
  match h d with
  | (k, hk) => (iter_fixed M.flow (M.start d) hf k).symm.trans hk

/-! ## V · the bridge -/

inductive Tri | tt | ff | bot deriving DecidableEq, Repr

/-- A carrier of a row's bit: its halted state is the row's property
in both directions. -/
structure Carrier (P : Prop) where
  terminal : Tri
  shadow : terminal = .bot ↔ P

theorem cannot_lie (P : Prop) (b : Carrier P) (h : b.terminal =
  .bot) : P := b.shadow.mp h

theorem cannot_deviate (P : Prop) (b : Carrier P) (t : P) :
  b.terminal = .bot := b.shadow.mpr t

/-- CANNOT BE MANUFACTURED: a halted carrier exists exactly when the
property holds. -/
theorem halted_iff (P : Prop) : (∃ b : Carrier P, b.terminal = .bot)
  ↔ P :=
  (fun (b, h) => b.shadow.mp h, fun t => ({.bot, {fun _ => t, fun _
    => rfl}}, rfl))

/-- The act's own carrier: halted on every frame where the act
holds. -/
def actCarrier (M : Frame) (h : Rests M) : Carrier (Value M) :=
  (.bot, {fun _ => (act_is_the_value M).mp h, fun _ => rfl})

/-- EVERY CARRIER IS THE ACT'S: under the act, every carrier of the
row's value is the act's own carrier. -/
theorem every_halted_carrier_is_the_act (M : Frame) (h : Rests M) :
  ∀ b : Carrier (Value M), b = actCarrier M h :=
  fun b => match b with
  | (.bot, _) => rfl
  | (.tt, s) => absurd (s.mpr ((act_is_the_value M).mp h)) (fun e =>
    Tri.noConfusion e)
  | (.ff, s) => absurd (s.mpr ((act_is_the_value M).mp h)) (fun e =>
    Tri.noConfusion e)

/-! ## VI · the Perelman witness -/

/-- The Perelman witness on a flow: a functional strictly decreasing
at every state off the seat,
and invariant under rescaling. -/
structure Witness (F : Flow) where
  W : F.State → Nat
  decrease : ∀ s, s ≠ F.round → W (F.step s) < W s
  scaleFree : ∀ s, W (F.rescale s) = W s

```

```

theorem witness_bound (F : Flow) (P : Witness F) :  $\forall$  n s, P.W s  $\leq$  n
   $\rightarrow$  Registers F s
| 0, s, hs => match F.deq s with
| isTrue e => (0, e)
| isFalse ne => absurd (Nat.lt_of_lt_of_le (P.decrease s ne) hs)
(Nat.not_lt_zero _)
| n + 1, s, hs => match F.deq s with
| isTrue e => (0, e)
| isFalse ne =>
  match witness_bound F P n (F.step s) (Nat.le_of_lt_succ
    (Nat.lt_of_lt_of_le (P.decrease s ne) hs)) with
  | (k, hk) => (k + 1, hk)

/-- THE WITNESS REGISTERS: a Perelman witness brings every state to
rest on the seat. -/
theorem witness_registers (F : Flow) (P : Witness F) (s : F.State) :
  Registers F s :=
  witness_bound F P (P.W s) s (Nat.le_refl _)

/-- A WITNESS SUPPLIES THE ACT: a witness on the row's flow brings
every manifold of the row to rest. -/
theorem witness_gives_act (M : Frame) (P : Witness M.flow) : Rests M
:=
  fun d => witness_registers M.flow P (M.start d)

/-- SUBTYPE OF THE BRIDGE: a witness yields a halted carrier of the
row's value. -/
def witnessCarrier (M : Frame) (P : Witness M.flow) : Carrier (Value
M) :=
  actCarrier M (witness_gives_act M P)

theorem witness_carrier_halted (M : Frame) (P : Witness M.flow) :
  (witnessCarrier M P).terminal = .bot := rfl

/-- A self-similar flow: the rescaling is the flow itself, and one
state lies off the seat and
steps onto it. -/
def solitonFlow : Flow := (Bool, fun _ => true, true, rfl, fun _ =>
  true, fun _ => rfl, rfl,
  fun s => match s with | true => isTrue rfl | false => isFalse
  Bool.noConfusion)

def solitonFrame : Frame := (Unit, solitonFlow, fun _ => false)

/-- STRICT SUBSET: the self-similar flow brings every manifold to
rest, so the act holds, and it
admits no Perelman witness, because a functional invariant under the
rescaling cannot strictly
decrease along a flow that is its own rescaling. -/
theorem act_without_witness : Rests solitonFrame  $\wedge$   $\neg$  Nonempty
  (Witness solitonFlow) :=
  (fun _ => (1, rfl),
  fun (P) =>
    have d : P.W true < P.W false := P.decrease false
    Bool.noConfusion
    have e : P.W (solitonFlow.rescale false) = P.W false :=
    P.scaleFree false
    have e' : P.W true = P.W false := e
    Nat.lt_irrefl _ (e'  $\triangleright$  d))

/-- NO WITNESS ON A SELF-SIMILAR FLOW: on every flow whose rescaling
is its step and which has a state off the seat,
no Perelman witness exists, since a functional invariant under the
rescaling cannot strictly decrease along the step
at that state. -/
theorem no_witness_on_self_similar (F : Flow) (h :  $\forall$  s, F.rescale s
= F.step s) (s : F.State)
(hs : s  $\neq$  F.round) :  $\neg$  Nonempty (Witness F) :=
  fun (P) =>
    have d : P.W (F.step s) < P.W s := P.decrease s hs
    have e : P.W (F.rescale s) = P.W s := P.scaleFree s
    have e' : P.W (F.step s) = P.W s := h s  $\triangleright$  e
    Nat.lt_irrefl _ (e'  $\triangleright$  d)

/-- THE WITNESS IS A STRICT SUBTYPE OF THE BRIDGE: on the
self-similar frame a halted carrier of the row's value
exists and no witness does. -/
theorem bridge_without_witness :
  ( $\exists$  b : Carrier (Value solitonFrame), b.terminal = .bot)  $\wedge$   $\neg$ 
  Nonempty (Witness solitonFlow) :=
  ((halted_iff (Value solitonFrame)).mpr ((act_is_the_value
  solitonFrame).mp act_without_witness.1),
  act_without_witness.2)

```

```

/-- A DOWNSTREAM STRICT SUBTYPE OF THE BRIDGE, downstream in one
sense, the factorization: every halted carrier of
the row's value is the bridge's carrier at the act, so whatever the
witness yields comes out of the bridge; where a
witness exists every carrier of the row's value halts with the
witness's carrier, which is the bridge's carrier at
the act the witness supplies; and the bridge holds a halted carrier
of the row's value on a frame where no witness
exists. -/
theorem witness_downstream_strict_subtype :
  ( $\forall$  (M : Frame) (P : Witness M.flow) (b : Carrier (Value M)),
  b.terminal = .bot  $\wedge$  b.terminal = (witnessCarrier M
  P).terminal)  $\wedge$ 
  ( $\forall$  (M : Frame) (P : Witness M.flow),
  witnessCarrier M P = actCarrier M (witness_gives_act M P)  $\wedge$ 
  (witnessCarrier M P).terminal = .bot)  $\wedge$ 
  ( $\exists$  b : Carrier (Value solitonFrame), b.terminal = .bot)  $\wedge$   $\neg$ 
  Nonempty (Witness solitonFlow) :=
  (fun M P b =>
    have e : b.terminal = .bot := cannot_deviate (Value M) b
    ((act_is_the_value M).mp (witness_gives_act M P))
    (e, e),
  fun _ => (rfl, rfl), bridge_without_witness)

/-- The crossing on a flow: a functional never rising along the
step, invariant under rescaling, and such that
a state off the seat where it holds level steps onto the seat. -/
structure Crossing (F : Flow) where
  W      : F.State  $\rightarrow$  Nat
  mono   :  $\forall$  s, W (F.step s)  $\leq$  W s
  scaleFree :  $\forall$  s, W (F.rescale s) = W s
  classify :  $\forall$  s, s  $\neq$  F.round  $\rightarrow$  W (F.step s) = W s  $\rightarrow$  F.step s =
    F.round

/-- THE CROSSING BOUND: a state whose crossing functional is at most
n comes to rest on the seat. -/
theorem crossing_bound (F : Flow) (C : Crossing F) :  $\forall$  n s, C.W s  $\leq$ 
  n  $\rightarrow$  Registers F s :=
  fun n => match n with
  | 0 => fun s hs => match F.deq s with
  | isTrue e => (0, e)
  | isFalse ne => match Nat.lt_or_ge (C.W (F.step s)) (C.W s) with
  | Or.inl lt => absurd (Nat.lt_of_lt_of_le lt hs)
  (Nat.not_lt_zero _)
  | Or.inr ge => (1, C.classify s ne (Nat.le_antisymm (C.mono s)
  ge))
  | n + 1 => fun s hs => match F.deq s with
  | isTrue e => (0, e)
  | isFalse ne => match Nat.lt_or_ge (C.W (F.step s)) (C.W s) with
  | Or.inl lt =>
    match crossing_bound F C n (F.step s) (Nat.le_of_lt_succ
    (Nat.lt_of_lt_of_le lt hs)) with
    | (k, hk) => (k + 1, hk)
  | Or.inr ge => (1, C.classify s ne (Nat.le_antisymm (C.mono s)
  ge))

/-- THE CROSSING REGISTERS: a crossing brings every state to rest on
the seat. -/
theorem crossing_registers (F : Flow) (C : Crossing F) (s : F.State)
: Registers F s :=
  crossing_bound F C (C.W s) s (Nat.le_refl _)

/-- Every witness is a crossing: a functional strictly decreasing
off the seat never rises and never holds
level off the seat. -/
def Witness.toCrossing {F : Flow} (P : Witness F) : Crossing F where
  W      := P.W
  mono   := fun s => match F.deq s with
  | isTrue e => by rw [e, F.fixed]; exact Nat.le_refl _
  | isFalse ne => Nat.le_of_lt (P.decrease s ne)
  scaleFree := P.scaleFree
  classify := fun s ne eq =>
    absurd (Nat.lt_of_lt_of_le (P.decrease s ne) (Nat.le_of_eq
    eq.symm)) (Nat.lt_irrefl _)

/-- The self-similar flow carries a crossing: the constant
functional. -/
def solitonCrossing : Crossing solitonFlow := (fun _ => 0, fun _ =>
  Nat.le_refl 0, fun _ => rfl, fun _ => rfl)

/-- THE CROSSING TYPED: every witness yields a crossing; a crossing
on the row's flow supplies the act; and the
self-similar flow carries a crossing and no witness, so the crossing
contains the witness strictly. -/

```

```

theorem crossing_types_the_crossing :
  (∀ F : Flow, Witness F → Nonempty (Crossing F)) ∧
  (∀ M : Frame, Crossing M.flow → Rests M) ∧
  Nonempty (Crossing solitonFlow) ∧ ¬ Nonempty (Witness
solitonFlow) :=
(fun _ P => (P.toCrossing), fun M C d => crossing_registers M.flow
C (M.start d), {solitonCrossing},
act_without_witness.2)

/-- A cycle: two states swapping, neither on the seat. -/
def cycleStep : Option Bool → Option Bool
| none => none
| some b => some (!b)

def cycleFlow : Flow := (Option Bool, cycleStep, none, rfl, id, fun
_ => rfl, rfl,
fun s => match s with
| none => isTrue rfl
| some _ => isFalse Option.noConfusion)

/-- NO WITNESS ON A CYCLE, WHERE THE ACT ALSO FAILS: on a cycle no
Perelman witness exists. -/
theorem no_witness_on_cycle : ¬ Nonempty (Witness cycleFlow) := fun
(P) =>
have a : P.W (show cycleFlow.State from some false) < P.W (show
cycleFlow.State from some true) :=
P.decrease (show cycleFlow.State from some true)
Option.noConfusion
have b : P.W (show cycleFlow.State from some true) < P.W (show
cycleFlow.State from some false) :=
P.decrease (show cycleFlow.State from some false)
Option.noConfusion
Nat.lt_irrefl _ (Nat.lt_trans a b)

/-- The cycle frame: one manifold starting on the cycle, off the
seat. -/
def cycleFrame : Frame := (Unit, cycleFlow, fun _ => (some true :
Option Bool))

/-- THE CYCLE STAYS OFF THE SEAT: from a state of the cycle the flow
is on the cycle at every time. -/
theorem cycle_stays_off : ∀ (k : Nat), ∃ c : Bool, iter
cycleStep k (some b) = some c :=
fun k => match k with
| 0 => fun b => (b, rfl)
| k + 1 => fun b => cycle_stays_off k (!b)

/-- THE CYCLE NEVER RESTS: on the cycle frame the act fails, and
with it the value. -/
theorem cycle_never_rests : ¬ Value cycleFrame := fun h => match h
() with
| (k, hk) => match cycle_stays_off k true with
| (_, hc) => Option.noConfusion (hc.symm.trans hk)

/-! ## VII · the proved part by dimension -/

/-- The generalized row by dimension: homotopy spheres, each
carrying its dimension (in dimension
three these are exactly the closed simply connected manifolds), and
each dimension's cited theorem
as a field: at most two (surfaces; zero and one degenerate), three
(the cited Ricci-flow theorem),
four (topological), five and above. -/
structure Dimensional where
M : Frame
dim : M.D → Nat
low : ∀ d, dim d ≤ 2 → Registers M.flow (M.start d)
three : ∀ d, dim d = 3 → Registers M.flow (M.start d)
four : ∀ d, dim d = 4 → Registers M.flow (M.start d)
high : ∀ d, 5 ≤ dim d → Registers M.flow (M.start d)

theorem dim_cases (n : Nat) : n ≤ 2 ∨ n = 3 ∨ n = 4 ∨ 5 ≤ n :=
match n with
| 0 => Or.inl (by decide) | 1 => Or.inl (by decide) | 2 => Or.inl
(by decide)
| 3 => Or.inr (Or.inl rfl) | 4 => Or.inr (Or.inr (Or.inl rfl))
| k + 5 => Or.inr (Or.inr (Or.inr (Nat.le_add_left 5 k)))

/-- EVERY DIMENSION DECIDED by its cited field. -/
theorem every_dimension_decided (S : Dimensional) : Value S.M := fun
d =>
match dim_cases (S.dim d) with
| Or.inl h => S.low d h
| Or.inr (Or.inl h) => S.three d h

```

```

| Or.inr (Or.inr (Or.inl h)) => S.four d h
| Or.inr (Or.inr (Or.inr h)) => S.high d h

structure DimensionalNoThree where
M : Frame
dim : M.D → Nat
low : ∀ d, dim d ≤ 2 → Registers M.flow (M.start d)
four : ∀ d, dim d = 4 → Registers M.flow (M.start d)
high : ∀ d, 5 ≤ dim d → Registers M.flow (M.start d)

def bareThree : DimensionalNoThree := (counter, fun _ => 3,
fun _ h => absurd h (by decide : ¬ ((3 : Nat) ≤ 2)), fun _ h =>
absurd h (by decide : ¬ ((3 : Nat) = 4)),
fun _ h => absurd h (by decide : ¬ (5 ≤ (3 : Nat))))

/-- THE DIMENSION-THREE FIELD IS LOAD-BEARING: without it a fake
three-sphere stands. -/
theorem three_is_load_bearing : bareThree.dim () = 3 ∧ ¬ Value
bareThree.M :=
(rfl, counter_fails)

/-! ## VIII · keyless and keyed -/

def Keyed (P : Frame → Prop) : Prop := ∃ M, ¬ P M

theorem value_is_keyed : Keyed Value := (counter, counter_fails)

/-- Keyed on both sides: the property holds on some frame and fails
on another. -/
def KeyedBoth (P : Frame → Prop) : Prop := (∃ M, P M) ∧ ∃ M, ¬ P M

/-- THE VALUE IS KEYED ON BOTH SIDES: it holds on the round world
and fails on the counter world. -/
theorem value_is_keyed_both : KeyedBoth Value := ({calm,
calm_value}, {counter, counter_fails})

/-- KEYED IS NOT KEYLESS: a property keyed on both sides is no
single statement read on every frame. -/
theorem keyed_is_not_keyless (P : Frame → Prop) (h : KeyedBoth P) :
¬ ∃ Q : Prop, ∀ M, (P M ↔ Q) :=
fun (_, hQ) => match h with
| ({M₁, p₁}, {M₂, n₂}) => n₂ ((hQ M₂).mpr ((hQ M₁).mp p₁))

theorem no_keyless_statement_is_the_act (P : Prop) : ¬ ∀ M : Frame,
(P ↔ Rests M) :=
fun h => given_is_not_the_value P (fun M => (h M).trans
(act_is_the_value M))

theorem pulse_does_not_certify : Root Empty (fun e => nomatch e) ∧ ¬
Value counter :=
(root_given, counter_fails)

/-! ## IX · freedom -/

/-- The kinetic record, the Bool model: one record for both values
of the bit. -/
def kineticRecord (_w : Bool) : Nat := 0

/-- THE RECORD WALL, the Bool model: no reading of the kinetic
record returns the bit. -/
theorem record_wall : ¬ ∃ g : Nat → Bool, ∀ w, g (kineticRecord w) =
w :=
fun (_, hg) => Bool.noConfusion ((hg false).symm.trans (hg true))

/-- THE RECORD WALL ON FRAMES: a record of frames that agrees on the
round world and the counter world admits no
reading that is the value on every frame. -/
theorem record_wall_on_frames {α : Type} (rec : Frame → α) (h : rec
calm = rec counter) :
¬ ∃ g : α → Prop, ∀ M : Frame, (g (rec M) ↔ Value M) :=
fun (_, hg) => counter_fails ((hg counter).mp (h ▶ (hg calm).mpr
calm_value))

theorem fibre_is_two : ({false, true}.filter (fun w => kineticRecord
w == kineticRecord true)).length = 2 := by
decide

def mulFibre (n : Nat) : List (Nat × Nat) :=
(List.range (n + 1)).flatMap fun a =>
(List.range (n + 1)).filterMap fun b => if a * b = n then some
(a, b) else none

theorem prime_shape :

```

```

mulFibre 7 = [(1, 7), (7, 1)] ∧ (mulFibre 7).all (fun p => p.1
!= p.2) = true := by
decide

/-! ## X · the triaxial lock -/

def dot2 (r x : Bool × Bool × Bool) : Bool :=
xor (r.1 && x.1) (xor (r.2.1 && x.2.1) (r.2.2 && x.2.2))

def cube : List (Bool × Bool × Bool) :=
[false, true].flatMap fun a => [false, true].flatMap fun b =>
[false, true].map fun c => (a, b, c)

def nonzero : List (Bool × Bool × Bool) := cube.filter (fun r => r
!= (false, false, false))

def solCount (rows : List ((Bool × Bool × Bool) × Bool)) : Nat :=
(cube.filter (fun x => rows.all (fun rt => dot2 rt.1 x ==
rt.2))).length

def xor3 (a b : Bool × Bool × Bool) : Bool × Bool × Bool :=
(xor a.1 b.1, xor a.2.1 b.2.1, xor a.2.2 b.2.2)

theorem two_axes_leave_two :
nonzero.all (fun r1 => nonzero.all (fun r2 => r1 == r2 ||
[false, true].all (fun t1 => [false, true].all (fun t2 =>
solCount [(r1, t1), (r2, t2)] == 2)))) = true := by decide

theorem three_axes_lock_one :
nonzero.all (fun r1 => nonzero.all (fun r2 => nonzero.all (fun
r3 =>
r1 == r2 || r3 == r1 || r3 == r2 || r3 == xor3 r1 r2 ||
[false, true].all (fun t1 => [false, true].all (fun t2 =>
[false, true].all (fun t3 =>
solCount [(r1, t1), (r2, t2), (r3, t3)] == 1)))))) = true :=
by decide

/-! ## XI · the record and the seed -/

/-- The world whose first n manifolds rest on the seat and whose
later ones are fake. -/
def staged (n : Nat) : Frame := (Nat, stuckFlow, fun d => if d < n
then true else false)

theorem finite_record_never_forces (n : Nat) :
(∀ d, d < n → Registers (staged n).flow ((staged n).start d)) ∧
¬ Value (staged n) :=
(fun d hd => {0, by show (if d < n then true else false) = true;
rw [if_pos hd],
fun h => match h n with
| (k, hk) => by
have e : (staged n).start n = false := by
show (if n < n then true else false) = false
rw [if_neg (Nat.lt_irrefl n)]
rw [e] at hk
exact Bool.noConfusion ((stuck_never k).symm.trans hk)})

/-- The world whose every manifold rests on the seat, on the staged
world's flow. -/
def allRest : Frame := (Nat, stuckFlow, fun _ => true)

/-- The resting world carries the value. -/
theorem allRest_value : Value allRest := fun _ => {0, rfl}

/-- THE FINITE RECORD WALL: the staged world agrees with the resting
world on its first n manifolds, and the value
holds on the resting world and fails on the staged one. -/
theorem finite_record_wall (n : Nat) :
(∀ d, d < n → (staged n).start d = allRest.start d) ∧ Value
allRest ∧ ¬ Value (staged n) :=
(fun d hd => by show (if d < n then true else false) = true; rw
[if_pos hd],
allRest_value, (finite_record_never_forces n).2)

structure Ladder where
Decided : Nat → Prop
base : Decided 0
mono : ∀ r s, Decided s → r ≤ s → Decided r
δ : Nat → Nat
step : ∀ r, Decided r → Decided (r + δ r)

theorem uniform_step_forces_all (L : Ladder) (hδ : ∀ r, 1 ≤ L.δ r) :
∀ r, L.Decided r
| 0 => L.base

```

```

| r + 1 => L.mono (r + 1) (r + L.δ r) (L.step r
(uniform_step_forces_all L hδ r))
(Nat.add_le_add_left (hδ r) r)

/-- The ladder on a flow's own enumerated states: rung r is decided
when the first r states register. -/
def rowLadder (F : Flow) (st : Nat → F.State) (δ : Nat → Nat)
(hstep : ∀ r, (∀ d, d < r → Registers F (st d)) → ∀ d, d < r + δ
r → Registers F (st d)) : Ladder :=
(fun r => ∀ d, d < r → Registers F (st d), fun d h => absurd h
(Nat.not_lt_zero d),
fun _ _ h h h h => h h (Nat.lt_of_lt_of_le h h h), δ, hstep)

/-- A UNIFORM STEP FORCES THE VALUE ON THE LADDER: a step of at
least one rung from every decided rung registers
every enumerated state. -/
theorem uniform_step_forces_value (F : Flow) (st : Nat → F.State) (δ
: Nat → Nat) (hδ : ∀ r, 1 ≤ δ r)
(hstep : ∀ r, (∀ d, d < r → Registers F (st d)) → ∀ d, d < r + δ
r → Registers F (st d)) :
∀ d, Registers F (st d) :=
fun d => uniform_step_forces_all (rowLadder F st δ hstep) hδ (d +
1) d (Nat.lt_succ_self d)

/-! ## XII · the closure, whole -/

theorem poincare_closure :
(∀ P : Prop, ¬ ∀ M : Frame, (P ↔ Value M)) ∧
(∀ M : Frame, Rests M ↔ Value M) ∧
(∀ A : ActualManifolds, Value A.M) ∧
(∀ (M : Frame) ( _ : Witness M.flow), Rests M) ∧
(Rests solitonFrame ∧ ¬ Nonempty (Witness solitonFlow)) ∧
¬ Nonempty (Witness cycleFlow) ∧
(∀ S : Dimensional, Value S.M) ∧
Keyed Value :=
(given_is_not_the_value, act_is_the_value,
poincare_from_existence, witness_gives_act,
act_without_witness, no_witness_on_cycle,
every_dimension_decided, value_is_keyed)

/-! ## The root, undeniable in deed -/

/-- A self-grounding root, the structure of Appendix R in the same
words: a type of acts, inhabited, every
one of which instances the root. -/
structure SelfGrounding (R : Prop) where
Act : Type
anAct : Act
instances : Act → R

/-- THE ROOT IS UNDENIABLE IN DEED: a denial of a self-grounding
root is a deed, and instances it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R)
(denial : G.Act) : R :=
G.instances denial

/-- No outside proof adds to a self-grounding root: an outside proof
of it is worth exactly the root. -/
theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R)
(Q : Prop) : (Q → R) ↔ R :=
(fun _ => G.instances G.anAct, fun r _ => r)

/-- THE UNDENIABLE ROOT HOLDS IN BOTH WORLDS: it is held by its deed
in a frame where the value holds
and in one where it fails, so it forces the value on no uniform
reading of the frames, and forces its failure on
none either. -/
theorem undeniable_root_forces_no_value {R : Prop} (G :
SelfGrounding R) :
R ∧ Value calm ∧ ¬ Value counter ∧ ¬ (∀ M : Frame, R → Value M) ∧
¬ (∀ M : Frame, R → ¬ Value M) :=
(G.instances G.anAct, calm_value, counter_fails, fun h =>
counter_fails (h counter (G.instances G.anAct)),
fun h => h calm (G.instances G.anAct) calm_value)

/-- The constructed root grounds itself: one act, which instances
it. -/
def constructedGrounds : SelfGrounding (Root Unit (fun _ => 1)) :=
(Unit, (), fun _ => constructed_root_holds)

/-- THE ROOT FORMULA IS NOT UNDENIABLE AS A FORMULA: with actuation
zero on the one existent it fails, so the root
holds by the deed that actuates, not by its shape. -/

```

```

theorem root_formula_is_not_undeniable : ¬ Root Unit (fun _ => 0) :=
  fun h => absurd (h ()) (show ¬ (0 : Int) < 0 by decide)

/-- THE ROOT READ ON THE ROW GROUNDS ITSELF EXACTLY WHERE THE VALUE
    HOLDS. -/
theorem root_on_row_grounds_iff (M : Frame) : Nonempty
  (SelfGrounding (RootOnRow M)) ↔ Value M :=
  (fun (G) => (root_on_row_is_the_value M).mp (G.instances G.anAct),
   fun h => ((Unit, ()), fun _ => (root_on_row_is_the_value M).mpr
    h)))

/-- THE ACT STANDS ON THE ROOT: the constructed root holds and
    forces the value on no uniform reading of the
    frames; read on the row it is the act; and the root read on the row
    grounds itself exactly where the act holds. -/
theorem the_act_stands_on_the_root :
  (Root Unit (fun _ => 1) ∧ ¬ (∀ M : Frame, Root Unit (fun _ => 1)
    → Value M)) ∧
  (∀ M : Frame, RootOnRow M ↔ Rests M) ∧
  (∀ M : Frame, Nonempty (SelfGrounding (RootOnRow M)) ↔ Rests M)
  :=
  ((constructed_root_holds, fun h => counter_fails (h counter
    constructed_root_holds)), root_on_row_is_the_act,
   fun M => (fun g => (act_is_the_value M).mpr
    ((root_on_row_grounds_iff M).mpr g),
     fun h => (root_on_row_grounds_iff M).mpr
    ((act_is_the_value M).mpr h)))

/-- THE POINCARÉ VALUE FROM THE ROOT READ ON THE ROW. -/
theorem poincare_from_root_on_row (M : Frame) (h : RootOnRow M) :
  Value M :=
  (root_on_row_is_the_value M).mp h

/-! ### the constructed root of Appendix R, repeated verbatim -/

/-- The actuation of Appendix R: one existent, whose actuation is
    one. -/
def ΔE₀ : Unit → Int := fun _ => 1

/-- The constructed root of Appendix R: every existent of the
    one-point type actuates. -/
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x

/-- The constructed root of Appendix R as a self-grounding root: one
    act, which instances it. -/
def ra₀ : SelfGrounding RA₀ := (Unit, ()), fun _ => show (0 : Int)
  < 1 by decide

/-- THE CONSTRUCTED ROOT OF APPENDIX R IS THE ROOT ON ONE EXISTENT
    with actuation one. -/
theorem RA₀_is_the_constructed_root : RA₀ ↔ Root Unit (fun _ => 1) :=
  Iff.rfl

/-- THE LOCK ON THE CONSTRUCTED ROOT: Appendix R's root holds,
    forces the value on no uniform reading of the frames,
    and read on the row of any frame is exactly that frame's value. -/
theorem the_lock_on_the_root :
  RA₀ ∧ ¬ (∀ M : Frame, RA₀ → Value M) ∧ (∀ M : Frame, RootOnRow M
    ↔ Value M) :=
  (ra₀.instances ra₀.anAct, (undeniable_root_forces_no_value
    ra₀).2.2.2.1, root_on_row_is_the_value)

/-- THE ROOT READ ON THE ROW IS KEYED: the root holds, and read
    uniformly on the frames it gives no value;
    read on the row, with the row's existents and the row's actuation,
    it is the value exactly, and keyed. -/
theorem root_read_on_row_is_keyed {R : Prop} (G : SelfGrounding R) :
  R ∧ ¬ (∀ M : Frame, R → Value M) ∧ (∀ M : Frame, RootOnRow M ↔
    Value M) ∧ (RootOnRow calm ∧ ¬ RootOnRow counter) :=
  (G.instances G.anAct, fun h => counter_fails (h counter
    (G.instances G.anAct)), root_on_row_is_the_value,
   root_on_row_is_keyed)

/-- THE LOCK, one theorem on no axiom: the constructed root holds
    with no hypothesis; every self-grounding
    root holds by its deed; no given statement is the value on every
    frame; the root read uniformly on the frames
    gives no value; read on the row the root is the act, the act is the
    value exactly, and both are keyed; the
    closing theorem gives the value on every actual frame; no reading of
    the kinetic record returns the bit; and
    no finite record forces the value. -/
theorem the_lock {R : Prop} (G : SelfGrounding R) :
  Root Unit (fun _ => 1) ∧

```

```

  R ∧
  (∀ P : Prop, ¬ ∀ M : Frame, (P ↔ Value M)) ∧
  ¬ (∀ M : Frame, R → Value M) ∧
  (∀ M : Frame, RootOnRow M ↔ Rests M) ∧
  (∀ M : Frame, Rests M ↔ Value M) ∧
  (RootOnRow calm ∧ ¬ RootOnRow counter) ∧
  (Rests calm ∧ ¬ Rests counter) ∧
  (∀ A : ActualManifolds, Value A.M) ∧
  (¬ ∃ g : Nat → Bool, ∀ w, g (kineticRecord w) = w) ∧
  (∀ n : Nat, (∀ d, d < n → Registers (staged n).flow ((staged
    n).start d)) ∧ ¬ Value (staged n)) :=
  (constructed_root_holds, G.instances G.anAct,
   given_is_not_the_value, (root_read_on_row_is_keyed G).2.1,
   root_on_row_is_the_act, act_is_the_value, root_on_row_is_keyed,
   act_is_keyed, poincare_from_existence, record_wall,
   finite_record_never_forces)

/-- FROM EXISTENCE ALONE, in the sense the title carries: the
    closing theorem consumes existence read on
    the row and nothing else; that reading is available on exactly the
    frames where the value holds; and no
    statement given without the reading is the value on every frame. One
    theorem, on no axiom. -/
theorem from_existence_alone :
  (∀ A : ActualManifolds, Value A.M) ∧
  (∀ F : Frame, Nonempty { A : ActualManifolds // A.M = F } ↔
    Value F) ∧
  (∀ P : Prop, ¬ ∀ F : Frame, (P ↔ Value F)) :=
  (poincare_from_existence, supply_iff, given_is_not_the_value)

end PCClose

/-! ## Cones, pinned as printed -/
/-- info: 'PCClose.seat_is_fixed' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.seat_is_fixed
/-- info: 'PCClose.calm_value' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.calm_value
/-- info: 'PCClose.stuck_never' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.stuck_never
/-- info: 'PCClose.counter_fails' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.counter_fails
/-- info: 'PCClose.root_given' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.root_given
/-- info: 'PCClose.root_conservative' does not depend on any axioms
  -/
#guard_msgs in #print axioms PCClose.root_conservative
/-- info: 'PCClose.arrow_given' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.arrow_given
/-- info: 'PCClose.freedom_given' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.freedom_given
/-- info: 'PCClose.given_is_not_the_value' does not depend on any
  axioms -/
#guard_msgs in #print axioms PCClose.given_is_not_the_value
/-- info: 'PCClose.arrow_forces_nothing' does not depend on any
  axioms -/
#guard_msgs in #print axioms PCClose.arrow_forces_nothing
/-- info: 'PCClose.given_in_both_worlds' does not depend on any
  axioms -/
#guard_msgs in #print axioms PCClose.given_in_both_worlds
/-- info: 'PCClose.act_is_the_value' does not depend on any axioms
  -/
#guard_msgs in #print axioms PCClose.act_is_the_value
/-- info: 'PCClose.act_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.act_is_keyed
/-- info: 'PCClose.poincare_from_existence' does not depend on any
  axioms -/
#guard_msgs in #print axioms PCClose.poincare_from_existence
/-- info: 'PCClose.supply_iff' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.supply_iff
/-- info: 'PCClose.nothing_escapes' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.nothing_escapes
/-- info: 'PCClose.fake_sphere_refutes' does not depend on any
  axioms -/
#guard_msgs in #print axioms PCClose.fake_sphere_refutes
/-- info: 'PCClose.rest_is_permanent' does not depend on any axioms
  -/
#guard_msgs in #print axioms PCClose.rest_is_permanent
/-- info: 'PCClose.cannot_lie' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.cannot_lie
/-- info: 'PCClose.cannot_deviate' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.cannot_deviate
/-- info: 'PCClose.halted_iff' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.halted_iff
/-- info: 'PCClose.witness_bound' does not depend on any axioms -/

```

```

#guard_msgs in #print axioms PCClose.witness_bound
/-- info: 'PCClose.witness_registers' does not depend on any axioms
-/
#guard_msgs in #print axioms PCClose.witness_registers
/-- info: 'PCClose.witness_gives_act' does not depend on any axioms
-/
#guard_msgs in #print axioms PCClose.witness_gives_act
/-- info: 'PCClose.witness_carrier_halted' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.witness_carrier_halted
/-- info: 'PCClose.act_without_witness' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.act_without_witness
/-- info: 'PCClose.no_witness_on_cycle' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.no_witness_on_cycle
/-- info: 'PCClose.dim_cases' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.dim_cases
/-- info: 'PCClose.every_dimension_decided' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.every_dimension_decided
/-- info: 'PCClose.three_is_load_bearing' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.three_is_load_bearing
/-- info: 'PCClose.value_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.value_is_keyed
/-- info: 'PCClose.no_keyless_statement_is_the_act' does not depend
on any axioms -/
#guard_msgs in #print axioms PCClose.no_keyless_statement_is_the_act
/-- info: 'PCClose.pulse_does_not_certify' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.pulse_does_not_certify
/-- info: 'PCClose.record_wall' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.record_wall
/-- info: 'PCClose.fibre_is_two' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.fibre_is_two
/-- info: 'PCClose.prime_shape' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.prime_shape
/-- info: 'PCClose.two_axes_leave_two' does not depend on any axioms
-/
#guard_msgs in #print axioms PCClose.two_axes_leave_two
/-- info: 'PCClose.three_axes_lock_one' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.three_axes_lock_one
/-- info: 'PCClose.finite_record_never_forces' does not depend on
any axioms -/
#guard_msgs in #print axioms PCClose.finite_record_never_forces
/-- info: 'PCClose.uniform_step_forces_all' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.uniform_step_forces_all
/-- info: 'PCClose.poincare_closure' does not depend on any axioms
-/
#guard_msgs in #print axioms PCClose.poincare_closure
/-- info: 'PCClose.denial_reenacts_root' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.denial_reenacts_root
/-- info: 'PCClose.external_proof_adds_nothing' does not depend on
any axioms -/
#guard_msgs in #print axioms PCClose.external_proof_adds_nothing
/-- info: 'PCClose.undeniable_root_forces_no_value' does not depend
on any axioms -/
#guard_msgs in #print axioms PCClose.undeniable_root_forces_no_value
/-- info: 'PCClose.root_read_on_row_is_keyed' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.root_read_on_row_is_keyed
/-- info: 'PCClose.the_lock' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.the_lock
/-- info: 'PCClose.from_existence_alone' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.from_existence_alone
/-- info: 'PCClose.indicator_pos' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.indicator_pos
/-- info: 'PCClose.root_satisfiable' does not depend on any axioms
-/
#guard_msgs in #print axioms PCClose.root_satisfiable
/-- info: 'PCClose.constructed_root_holds' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.constructed_root_holds
/-- info: 'PCClose.root_on_row_is_the_act' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.root_on_row_is_the_act
/-- info: 'PCClose.root_on_row_is_the_value' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.root_on_row_is_the_value

```

```

/-- info: 'PCClose.root_on_row_is_keyed' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.root_on_row_is_keyed
/-- info: 'PCClose.act_is_the_weakest_premise' does not depend on
any axioms -/
#guard_msgs in #print axioms PCClose.act_is_the_weakest_premise
/-- info: 'PCClose.existence_as_given_in_both_worlds' does not
depend on any axioms -/
#guard_msgs in #print axioms
PCClose.existence_as_given_in_both_worlds
/-- info: 'PCClose.nothing_given_refutes' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.nothing_given_refutes
/-- info: 'PCClose.no_witness_on_self_similar' does not depend on
any axioms -/
#guard_msgs in #print axioms PCClose.no_witness_on_self_similar
/-- info: 'PCClose.bridge_without_witness' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.bridge_without_witness
/-- info: 'PCClose.witness_downstream_strict_subtype' does not
depend on any axioms -/
#guard_msgs in #print axioms
PCClose.witness_downstream_strict_subtype
/-- info: 'PCClose.iter_fixed' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.iter_fixed
/-- info: 'PCClose.nothing_given_forces' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.nothing_given_forces
/-- info: 'PCClose.the_one_state_never_left' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.the_one_state_never_left
/-- info: 'PCClose.every_halted_carrier_is_the_act' does not depend
on any axioms -/
#guard_msgs in #print axioms
PCClose.every_halted_carrier_is_the_act
/-- info: 'PCClose.crossing_bound' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.crossing_bound
/-- info: 'PCClose.crossing_registers' does not depend on any axioms
-/
#guard_msgs in #print axioms PCClose.crossing_registers
/-- info: 'PCClose.crossing_types_the_crossing' does not depend on
any axioms -/
#guard_msgs in #print axioms PCClose.crossing_types_the_crossing
/-- info: 'PCClose.cycle_stays_off' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.cycle_stays_off
/-- info: 'PCClose.cycle_never_rests' does not depend on any axioms
-/
#guard_msgs in #print axioms PCClose.cycle_never_rests
/-- info: 'PCClose.value_is_keyed_both' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.value_is_keyed_both
/-- info: 'PCClose.keyed_is_not_keyless' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.keyed_is_not_keyless
/-- info: 'PCClose.record_wall_on_frames' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.record_wall_on_frames
/-- info: 'PCClose.allRest_value' does not depend on any axioms -/
#guard_msgs in #print axioms PCClose.allRest_value
/-- info: 'PCClose.finite_record_wall' does not depend on any axioms
-/
#guard_msgs in #print axioms PCClose.finite_record_wall
/-- info: 'PCClose.uniform_step_forces_value' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.uniform_step_forces_value
/-- info: 'PCClose.root_formula_is_not_undeniable' does not depend
on any axioms -/
#guard_msgs in #print axioms PCClose.root_formula_is_not_undeniable
/-- info: 'PCClose.root_on_row_grounds_iff' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.root_on_row_grounds_iff
/-- info: 'PCClose.the_act_stands_on_the_root' does not depend on
any axioms -/
#guard_msgs in #print axioms PCClose.the_act_stands_on_the_root
/-- info: 'PCClose.poincare_from_root_on_row' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.poincare_from_root_on_row
/-- info: 'PCClose.RA0 is the constructed root' does not depend on
any axioms -/
#guard_msgs in #print axioms PCClose.RA0 is the constructed root
/-- info: 'PCClose.the_lock_on_the_root' does not depend on any
axioms -/
#guard_msgs in #print axioms PCClose.the_lock_on_the_root

```

APPENDIX R • THE ROOT ON NO AXIOM AND NO POSIT

The root on which this paper stands is proved here, in this paper, so that no reader needs another document to check it. The kernel below, R.1, TOE_Zero.lean, is carried verbatim from the root paper (Islam 2026i). Compiled on Lean 4.19.0 and on Lean 4.22.0 it exits 0 with no message: it declares no axiom, imports nothing, and every one of its fifteen theorems prints *does not depend on any axioms*, pinned at the foot of the listing. Five declarations carry its root sections, SelfGrounding, SelfVerifying, ΔE_0 , RA_0 and ra_0 , and all five stand in the listing; the algebra of its master seal rests on the remaining definitions of the listing, the quaternion chart among them. A root grounds itself when deeds occur and every deed instances it (SelfGrounding); the root kernel's own comments call a deed an act. The constructed domain has one existent, whose energy of actuation is one, so the Root Axiom there reads $\forall x, 0 < \Delta E_0(x) (RA_0)$. On that domain the Root Axiom is a theorem with no axiom and no hypothesis (root_on_the_constructed_domain). For every root that grounds itself, the root holds, every deed re-enacts it, and no outside proof adds anything to it, one closed theorem on no axiom (the_floor_is_universal). Every denial of the root is a deed that instances it (denial_reenacts_root); the root is held by its deed (seated_undeniable); it is self-verifying (denial_instantiates); and no level stands above it (no_level_above). The master seal binds the root's universal law and the constructed root with the Return, the scalar line as the fixed set of conjugation on the chart, and the twelve gates counted three ways, as the Hurwitz units up to sign, as the norm-two shell up to sign and as the even permutations of four points with class equation 1, 3, 4, 4, in one theorem on no axiom (the_master_seal); it states the three counts side by side and no map between them. The body of this paper reads the root on its row; this appendix is the root itself. The row kernel repeats the constructed root verbatim, proves it is the root on one existent with actuation one, and locks it to the row (RA_0 is the constructed root, the_lock_on_the_root). Its SHA-256 is

```
0cfcd9f6b399ecbaef01e762a4d9f104
90288f1053827b404428062e55076238
```

```
/-
TOE_Zero.lean • Theory of Theories of Everything (TOE of All
TOEs): Existence Proves Existence Only by Motion
The zero-axiom, zero-posit kernel. Core Lean 4: no import, no
axiom declared, no sorry, no native_decide, and
every theorem rests on no axiom at all, not even propositional
extensionality, quotients or choice.
I • The self-grounding root. Acts occur and every act instances
the root. Then every denial is an act and
re-enacts the root, the root is held by the act itself, and no
external proof adds anything to it: the
Omega Boundary, proved for every root that grounds itself,
universally, by quantification.
II • The constructed root. On the constructed domain the Root
Axiom is a theorem.
```

```
III • The Return and the line. On the integer quaternions i·j·k =
-1 and each unit squares to -1; conjugation is
an involution and its fixed set on the chart is exactly the
scalar line.
IV • The twelve gates, recovered three ways inside the integers of
the quaternions: the twenty-four Hurwitz units
counted twelve up to sign; the twelve even permutations with
class equation 1, 3, 4, 4; and the norm-two
shell of twenty-four, split twelve and twelve.
V • The regress terminated. Above a self-grounding root no level
grounds it further.
VI • The master seal, every result bound in one theorem.
-/
namespace TOE0

/-- ## I • The self-grounding root: the Omega Boundary -/

/-- A self-grounding root: acts occur, and every act instances the
root. -/
structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct     : Act
  instances : Act → R

/-- A proposition whose denial hands it over. -/
def SelfVerifying (P : Prop) : Prop := ¬P → P

/-- THE OMEGA BOUNDARY: every denial of the root is an act, and
re-enacts it. -/
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R)
  (denial : G.Act) : R := G.instances denial

/-- EXISTENCE PROVES EXISTENCE ONLY BY MOTION: the root is held by
the act itself. -/
theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R :=
  G.instances G.anAct

/-- No external proof adds anything to the root. -/
theorem external_proof_adds_nothing {R : Prop} (G : SelfGrounding R)
  (Q : Prop) : (Q → R) → R :=
  (fun _ => G.instances G.anAct, fun r _ => r)

/-- The deed of denying the root hands it over. -/
theorem denial_instantiates {R : Prop} (G : SelfGrounding R) :
  SelfVerifying R := fun _ => G.instances G.anAct

/-- THE FLOOR IS UNIVERSAL: for every root that grounds itself, the
root holds, every act re-enacts it, and no
external proof adds to it. -/
theorem the_floor_is_universal :
  ∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q :
  Prop, (Q → R) → R) :=
  fun _ G => (seated_undeniable G, denial_reenacts_root G,
  external_proof_adds_nothing G)

/-! ## II • The constructed root -/

/-- The Root Axiom on the constructed one-point domain: its one
existent actuates, at positive energy. -/
def ΔE0 : Unit → Int := fun _ => 1
def RA0 : Prop := ∀ x : Unit, 0 < ΔE0 x

/-- The constructed domain grounds its root: its one act is a deed,
and the deed actuates. -/
def ra0 : SelfGrounding RA0 := (Unit, (), fun _ _ => show (0 : Int)
< 1 by decide)

/-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM. -/
theorem root_on_the_constructed_domain : RA0 := seated_undeniable
ra0

/-! ## III • The Return and the line -/

structure Q where
  (a b c d : Int)
  deriving DecidableEq

def Q.mul (x y : Q) : Q :=
  (x.a * y.a - x.b * y.b - x.c * y.c - x.d * y.d, x.a * y.b + x.b *
  y.a + x.c * y.d - x.d * y.c,
  x.a * y.c - x.b * y.d + x.c * y.a + x.d * y.b, x.a * y.d + x.b *
  y.c - x.c * y.b + x.d * y.a)

def Q.conj (x : Q) : Q := (x.a, -x.b, -x.c, -x.d)
```

```

def qone : Q := (1, 0, 0, 0)
def qneg : Q := (-1, 0, 0, 0)
def qi : Q := (0, 1, 0, 0)
def qj : Q := (0, 0, 1, 0)
def qk : Q := (0, 0, 0, 1)

/-- THE RETURN: i.j.k = -1. -/
theorem the_return : (qi.mul qj).mul qk = qneg := by decide

/-- Each unit squares to -1. -/
theorem units_square_to_minus_one : qi.mul qi = qneg ∧ qj.mul qj =
  qneg ∧ qk.mul qk = qneg := by decide

/-- The chart: every quaternion with coordinates in -2 .. 2. -/
def chart : List Int := [-2, -1, 0, 1, 2]
def quats : List Q :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c
    => chart.map fun d => (a, b, c, d)

set_option maxRecDepth 100000 in
/-- Conjugation is an involution on the chart. -/
theorem conj_involutive_on_the_chart : quats.all (fun q =>
  q.conj.conj == q) = true := by decide

set_option maxRecDepth 100000 in
/-- THE LINE: conjugation fixes exactly the scalar line on the
  chart. -/
theorem the_line_is_the_fixed_set :
  quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 &&
    q.d == 0)) = true := by decide

/-! ## IV · The twelve gates, recovered three ways -/

/-- Hurwitz quaternions in doubled coordinates: all even, or all
  odd. -/
def hurwitz (a b c d : Int) : Bool :=
  (a % 2 == 0 && b % 2 == 0 && c % 2 == 0 && d % 2 == 0) || (a % 2 !=
    0 && b % 2 != 0 && c % 2 != 0 && d % 2 != 0)
/-- Doubled coordinates -2 .. 2 hold every unit (doubled norm four)
  and every element of the norm-two shell (doubled
  norm eight), since a doubled coordinate of absolute value three
  or more already exceeds both norms. -/
def doubled : List (Int × Int × Int × Int) :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c
    => chart.map fun d => (a, b, c, d)

/-- The Hurwitz units: norm one, doubled norm four. -/
def units : List (Int × Int × Int × Int) :=
  doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d ==
    4 && hurwitz a b c d

/-- The first nonzero coordinate is positive: one representative of
  each pair ±u. -/
def upToSign (p : Int × Int × Int × Int) : Bool :=
  let (a, b, c, d) := p
  if a != 0 then 0 < a else if b != 0 then 0 < b else if c != 0 then
    0 < c else 0 < d

set_option maxRecDepth 100000 in
/-- FIRST WAY: twenty-four Hurwitz units, twelve up to sign. -/
theorem twenty_four_units_twelve_gates : units.length = 24 ∧
  (units.filter upToSign).length = 12 := by decide

/-- The norm-two shell: doubled norm eight. -/
def shell : List (Int × Int × Int × Int) :=
  doubled.filter fun (a, b, c, d) => a * a + b * b + c * c + d * d ==
    8 && hurwitz a b c d

set_option maxRecDepth 100000 in
/-- THIRD WAY: the norm-two shell has twenty-four elements, split
  twelve and twelve by sign. -/
theorem the_shell_splits_twelve_and_twelve :
  shell.length = 24 ∧ (shell.filter upToSign).length = 12 ∧
  (shell.filter (fun p => !upToSign p)).length = 12 := by
  decide

/-- Permutations of four points, as their lists of images. -/
def perms4 : List (List Nat) :=
  [0, 1, 2, 3].flatMap fun a => [0, 1, 2, 3].flatMap fun b => [0, 1,
    2, 3].flatMap fun c =>
    [0, 1, 2, 3].filterMap fun d => if [a, b, c, d].eraseDups.length
      == 4 then some [a, b, c, d] else none

def inversions (p : List Nat) : Nat :=
  ((List.range 4).flatMap fun i => (List.range 4).map fun j => (i,
    j)).countP
    fun (i, j) => i < j && p.getD j 0 < p.getD i 0

/-- The twelve gates as the even permutations. -/
def gates : List (List Nat) := perms4.filter fun p => inversions p %
  2 == 0

def compose (p q : List Nat) : List Nat := q.map fun i => p.getD i 0
def inverse (p : List Nat) : List Nat := [0, 1, 2, 3].map fun i =>
  p.idxOf i

def conjClass (g : List Nat) : List (List Nat) := (gates.map fun h
  => compose (compose h g) (inverse h)).eraseDups

def insertSorted (n : Nat) : List Nat → List Nat
  | [] => [n]
  | m :: ms => if n ≤ m then n :: m :: ms else m :: insertSorted n ms

def sortNat (l : List Nat) : List Nat := l.foldr insertSorted []

/-- The class equation from the sorted class size of each element: a
  class of size s contributes s elements. -/
def collapse : Nat → List Nat → List Nat
  | 0, _ => []
  | _ + 1, [] => []
  | fuel + 1, s :: rest => s :: collapse fuel (rest.drop (s - 1))

set_option maxRecDepth 100000 in
/-- SECOND WAY: the twelve gates, with class equation 1, 3, 4, 4. -/
theorem the_class_equation :
  gates.length = 12 ∧ collapse 12 (sortNat (gates.map fun g =>
    (conjClass g).length)) = [1, 3, 4, 4] := by
  decide

/-! ## V · The regress terminated -/

/-- THE REGRESS ENDS AT THE ROOT: any proposition offered as a
  further ground of the root is idle; the root needs
  no level above it. -/
theorem no_level_above {R : Prop} (G : SelfGrounding R) : ∀ Q :
  Prop, (Q → R) → R := fun _ => G.instances G.anAct

/-! ## VI · The master seal -/

/-- THE MASTER SEAL: the floor is universal; the root is a theorem
  on the constructed domain; the Return closes;
  the line is the fixed set; and the twelve gates are recovered
  three ways. -/
theorem the_master_seal :
  (∀ (R : Prop) (G : SelfGrounding R), R ∧ (∀ _ : G.Act, R) ∧ (∀ Q
    : Prop, (Q → R) → R)) ∧
  RA₀ ∧ (qi.mul qj).mul qk = qneg ∧
  quats.all (fun q => (q.conj == q) == (q.b == 0 && q.c == 0 &&
    q.d == 0)) = true ∧
  (units.filter upToSign).length = 12 ∧ gates.length = 12 ∧
  (shell.filter upToSign).length = 12 ∧
  collapse 12 (sortNat (gates.map fun g => (conjClass g).length))
    = [1, 3, 4, 4] :=
  (the_floor_is_universal, root_on_the_constructed_domain,
    the_return, the_line_is_the_fixed_set,
    twenty_four_units_twelve_gates.2, the_class_equation.1,
    the_shell_splits_twelve_and_twelve.2.1, the_class_equation.2)

end TOE0

/-! ## The zero law, pinned: every cone is empty -/
/-- info: 'TOE0.denial_reenacts_root' does not depend on any axioms
  -/
#guard_msgs in #print axioms TOE0.denial_reenacts_root
/-- info: 'TOE0.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.seated_undeniable
/-- info: 'TOE0.external_proof_adds_nothing' does not depend on any
  axioms -/
#guard_msgs in #print axioms TOE0.external_proof_adds_nothing
/-- info: 'TOE0.denial_instantiates' does not depend on any axioms
  -/
#guard_msgs in #print axioms TOE0.denial_instantiates
/-- info: 'TOE0.the_floor_is_universal' does not depend on any
  axioms -/
#guard_msgs in #print axioms TOE0.the_floor_is_universal
/-- info: 'TOE0.root_on_the_constructed_domain' does not depend on
  any axioms -/
#guard_msgs in #print axioms TOE0.root_on_the_constructed_domain

```

```

/-- info: 'TOE0.the_return' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_return
/-- info: 'TOE0.units_square_to_minus_one' does not depend on any
axioms -/
#guard_msgs in #print axioms TOE0.units_square_to_minus_one
/-- info: 'TOE0.conj_involutive_on_the_chart' does not depend on any
axioms -/
#guard_msgs in #print axioms TOE0.conj_involutive_on_the_chart
/-- info: 'TOE0.the_line_is_the_fixed_set' does not depend on any
axioms -/
#guard_msgs in #print axioms TOE0.the_line_is_the_fixed_set
/-- info: 'TOE0.twenty_four_units_twelve_gates' does not depend on
any axioms -/
#guard_msgs in #print axioms TOE0.twenty_four_units_twelve_gates
/-- info: 'TOE0.the_shell_splits_twelve_and_twelve' does not depend
on any axioms -/
#guard_msgs in #print axioms TOE0.the_shell_splits_twelve_and_twelve
/-- info: 'TOE0.the_class_equation' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_class_equation
/-- info: 'TOE0.no_level_above' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.no_level_above
/-- info: 'TOE0.the_master_seal' does not depend on any axioms -/
#guard_msgs in #print axioms TOE0.the_master_seal

```

The root's axes are proved in the second kernel, R.2, Triaxial_Actuation.lean (32 theorems, every one on no axiom; SHA-256 566615a6b9610fc981e03eb42e07af9e03849f20704729880f2bfe6434f2ea0b). For every actuation, the direction of its deed gives the fold, the registration and the seat; the registration is the only height-keeping map onto the seat; the seat is the only place both blind spots cancel; no reading of the record returns the orientation; and every self-grounding root is the root of an actuation over its own deeds, so the root carries the three axes and is not characterless (actuation_is_tri_axial, registration_is_forced, the_only_special_cut, tongue_freedom, every_root_actuates, the_root_carries_three_axes). The root's grade is read in the third kernel, R.3, Root_Grade_Ledger.lean (18 theorems, every one on no axiom; SHA-256 c18b68b29d2b6be5058d56d3a7f9e966bcde6ce099217e0f5945db4c829c42b8). On the grade ledger of the Master Codex (Islam 2026l), carried with its weakest-link join law, the Codex's rule reads a warranted root, one with a proof of its statement, at theorem grade, and an unwarranted root, the Codex's own declared axiom, at premise grade; the root of R.1 is warranted by a theorem with no axiom and no hypothesis, so on that rule it stands at theorem grade, the grade is read from the evidence the root is entered with, a warrant reading theorem and a posit premise, and not elected, and RA-Tongue co-location inherits the root's grade exactly (a_warranted_root_is_theorem_grade, the_ledger_reads_the_warrant, the_grade_is_not_elected, ra_is_theorem_grade, coloc_grade_is_the_roots_grade, the_ledger_seal). Both kernels compile with exit 0 and no message on Lean 4.19.0 and on Lean 4.22.0, declare no axiom, import nothing, and pin every cone as *does not depend on any axioms*; both are carried from the master plan of the series, Seven Rows, One Root (Islam 2026k), each extended in this revision of the series by one additive section, R.2 by its section IX and R.3 by its section VII, carried alike by every paper of the series, and the

paper that reads them on its row cites them for exactly what they state.

```

/-
Triaxial_Actuation.lean · actuation is triaxial

Core Lean 4: no import, no axiom declared, no sorry, no
native_decide, and every theorem rests on no
axiom at all, not even propositional extensionality, quotients or
choice.

An actuation is a self-grounding root whose acts carry a
direction: each act has a side and a height,
and reversing the deed reverses the side and keeps the height.
From the direction alone the kernel
constructs three axes and proves them forced:
I The fold: reversing the deed is an involution.
II The registration: it keeps the height and forgets the side;
it is blind to the fold, closed with
it under combination, and it is the only height-keeping map
onto the seat.
III The seat: the fold's fixed set, where the fold and the
registration both fix the act, and off
which neither does.
IV The third axis: off the seat the fold's orbit has two acts
with one record, so the record leaves
exactly the orientation open; every reading blind to the fold
reads both alike.
V The root through the axes: a reversed deed is still an act
and instances the root; the axes are
carried by every actuation, and they force no orientation.
VI The bridge atom: the integers, with negation as the fold, are
an actuation.
VII The triaxial seal.
VIII The Tongue: no reading of the record returns the orientation;
the seat is the only special cut.
IX Every root actuates: a self-grounding root is the root of an
actuation over its own acts, so the
root carries the three axes; the registration is the fold's
record and lands on the seat, where
the orientation an act and its reversal share stays open; the
even pair leaves two acts, the
orientation locks one; and a degenerate direction, one side
only, is the characterless case.
-/

namespace TriAct

/-! ## The root, self-grounding -/

/-- A self-grounding root: acts occur, and every act instances the
root. -/
structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct    : Act
  instances : Act → R

/-! ## Actuation: a self-grounding root whose acts carry a direction
-/

/-- The direction of a deed: a side with a reversal, whose one fixed
side is the zero side. -/
structure Direction where
  Side      : Type
  rev       : Side → Side
  zero      : Side
  rev_rev   : ∀ s, rev (rev s) = s
  rev_zero  : rev zero = zero
  fix_zero  : ∀ s, rev s = s → s = zero

/-- An actuation: a self-grounding root whose acts are pairs of a
side and a height. -/
structure Actuation (R : Prop) where
  dir      : Direction
  Height    : Type
  anAct     : dir.Side × Height
  instances : dir.Side × Height → R

variable {R : Prop}

/-- Every actuation is a self-grounding root. -/
def Actuation.toSelfGrounding (A : Actuation R) : SelfGrounding R :=
  (A.dir.Side × A.Height, A.anAct, A.instances)

```

```

/-- The fold: reversing the deed reverses the side and keeps the
height. -/
def fold (A : Actuation R) (p : A.dir.Side × A.Height) : A.dir.Side
× A.Height := (A.dir.rev p.1, p.2)

/-- The registration: it keeps the height and forgets the side. -/
def reg (A : Actuation R) (p : A.dir.Side × A.Height) : A.dir.Side ×
A.Height := (A.dir.zero, p.2)

/-- The seat: the acts the fold fixes. -/
def OnSeat (A : Actuation R) (p : A.dir.Side × A.Height) : Prop :=
fold A p = p

/-! ## I · the fold -/

theorem fold_involutive (A : Actuation R) (p : A.dir.Side ×
A.Height) : fold A (fold A p) = p := by
cases p with
| mk s h => show (A.dir.rev (A.dir.rev s), h) = (s, h); rw
[A.dir.rev_rev]

/-! ## II · the registration -/

/-- The registration forgets the side: an act and its reversal have
one record. -/
theorem reg_forgets_side (A : Actuation R) (p : A.dir.Side ×
A.Height) : reg A (fold A p) = reg A p := rfl

/-- The registration keeps the height. -/
theorem reg_keeps_height (A : Actuation R) (p : A.dir.Side ×
A.Height) : (reg A p).2 = p.2 := rfl

/-- The even pair is closed under combination: folding a record
changes nothing. -/
theorem fold_of_reg (A : Actuation R) (p : A.dir.Side × A.Height) :
fold A (reg A p) = reg A p := by
show (A.dir.rev A.dir.zero, p.2) = (A.dir.zero, p.2); rw
[A.dir.rev_zero]

/-- The registration is idempotent. -/
theorem reg_idempotent (A : Actuation R) (p : A.dir.Side × A.Height)
: reg A (reg A p) = reg A p := rfl

/-- THE REGISTRATION IS FORCED, NOT ELECTED: any map that keeps the
height and lands on the seat is
the registration. -/
theorem registration_is_forced (A : Actuation R) (g : A.dir.Side ×
A.Height → A.dir.Side × A.Height)
(keeps : ∀ p, (g p).2 = p.2) (lands : ∀ p, OnSeat A (g p)) : ∀ p,
g p = reg A p := by
intro p
have hs : (g p).1 = A.dir.zero := by
have hf := lands p
have h1 : A.dir.rev (g p).1 = (g p).1 := congrArg Prod.fst hf
exact A.dir.fix_zero _ h1
have hh := keeps p
cases hg : g p with
| mk s h =>
rw [hg] at hs hh
show (s, h) = (A.dir.zero, p.2)
rw [show s = A.dir.zero from hs, show h = p.2 from hh]

/-! ## III · the seat -/

/-- The seat is exactly the zero side. -/
theorem seat_iff_zero_side (A : Actuation R) (p : A.dir.Side ×
A.Height) : OnSeat A p ↔ p.1 = A.dir.zero :=
(fun h => A.dir.fix_zero _ (congrArg Prod.fst h),
fun h => by
cases p with
| mk s t =>
show (A.dir.rev s, t) = (s, t)
have hs : s = A.dir.zero := h
rw [hs, A.dir.rev_zero])

/-- ON THE SEAT BOTH BLIND SPOTS CANCEL: the fold and the
registration both fix the act. -/
theorem seat_cancels_both (A : Actuation R) (p : A.dir.Side ×
A.Height) (h : OnSeat A p) :
fold A p = p ∧ reg A p = p := by
refine (h, ?_)
have hz : p.1 = A.dir.zero := (seat_iff_zero_side A p).mp h
cases p with
| mk s t =>

```

```

have hs : s = A.dir.zero := hz
show (A.dir.zero, t) = (s, t)
rw [hs]

/-- OFF THE SEAT BOTH STAND: neither the fold nor the registration
fixes the act. -/
theorem off_seat_both (A : Actuation R) (p : A.dir.Side × A.Height)
(h : ¬ OnSeat A p) :
fold A p ≠ p ∧ reg A p ≠ p := by
refine (h, fun hr => h ?_)
have hz : p.1 = A.dir.zero := (congrArg Prod.fst hr).symm
exact (seat_iff_zero_side A p).mpr hz

/-! ## IV · the third axis -/

/-- THE RECORD LEAVES EXACTLY THE ORIENTATION: off the seat, an act
and its reversal are two acts with
one record. -/
theorem record_leaves_the_orientation (A : Actuation R) (p :
A.dir.Side × A.Height) (h : ¬ OnSeat A p) :
fold A p ≠ p ∧ reg A (fold A p) = reg A p :=
(h, rfl)

/-- Every reading blind to the fold reads an act and its reversal
alike. -/
theorem even_readings_are_blind (A : Actuation R) {α : Type} (f :
A.dir.Side × A.Height → α)
(blind : ∀ p, f (fold A p) = f p) (p : A.dir.Side × A.Height) :
f (fold A p) = f p :=
blind p

/-- The registration is such a reading, and so is every reading of
the record. -/
theorem readings_of_the_record_are_blind (A : Actuation R) {α :
Type} (k : A.dir.Side × A.Height → α)
(p : A.dir.Side × A.Height) : k (reg A (fold A p)) = k (reg A p)
:= rfl

/-! ## V · the root through the axes -/

/-- A REVERSED DEED IS STILL AN ACT, AND INSTANCES THE ROOT: the
denial spends the deed. -/
theorem denial_instances_root (A : Actuation R) (p : A.dir.Side ×
A.Height) : R :=
A.instances (fold A p)

/-- The root holds on every actuation, by its act. -/
theorem root_holds (A : Actuation R) : R := A.instances A.anAct

/-- THE AXES FORCE NO ORIENTATION: off the seat both orientations
are acts, both instance the root, and
the record reads them alike; the triaxial structure leaves the one
orientation open. -/
theorem axes_force_no_orientation (A : Actuation R) (p : A.dir.Side
× A.Height) (h : ¬ OnSeat A p) :
R ∧ R ∧ fold A p ≠ p ∧ reg A (fold A p) = reg A p :=
(A.instances p, A.instances (fold A p), h, rfl)

/-! ## VI · the bridge atom: the integers are an actuation -/

theorem int_neg_neg : ∀ s : Int, ¬(s) = s
| Int.ofNat 0 => rfl
| Int.ofNat (_ + 1) => rfl
| Int.negSucc _ => rfl

theorem self_neg_zero : ∀ v : Int, ¬v = v → v = 0
| Int.ofNat 0, _ => rfl
| Int.ofNat (_ + 1), h => Int.noConfusion h
| Int.negSucc _, h => Int.noConfusion h

/-- The integer direction: negation, with zero its one fixed side. -/
def intDir : Direction := (Int, fun s => -s, 0, int_neg_neg, rfl,
self_neg_zero)

/-- The bridge atom: points (d, t), the fold (d, t) ↔ (¬d, t), the
registration (d, t) ↔ (0, t). -/
def atom : Actuation True := (intDir, Int, ((0 : Int), (0 : Int)),
fun _ => True.intro)

/-- THE BRIDGE ATOM, AS AN ACTUATION: the fold fixes (d, t) exactly
when d = 0, the registration
leaves it unchanged exactly then, and off the seat the record leaves
the orientation. -/
theorem the_bridge_atom (d t : Int) :

```

```

(OnSeat atom (d, t) ↔ d = 0) ∧ (reg atom (d, t) = (d, t) ↔ d = 0)
∧
(d ≠ 0 → fold atom (d, t) ≠ (d, t) ∧ reg atom (fold atom (d, t))
= reg atom (d, t)) :=
(seat_iff_zero_side atom (d, t),
(fun h => (congrArg Prod.fst h).symm, fun h => by rw [h]; rfl),
fun hd => record_leaves_the_orientation atom (d, t) (fun hs => hd
((seat_iff_zero_side atom (d, t)).mp hs)))

/-- The atom is not characterless: off the seat it has two acts with
one record. -/
theorem the_atom_is_not_characterless :
fold atom ((1 : Int), (0 : Int)) ≠ ((1 : Int), (0 : Int)) ∧ reg
atom (fold atom ((1 : Int), (0 : Int))) = reg atom ((1 : Int), (0
: Int)) :=
(fun h => Int.noConfusion (congrArg Prod.fst h), rfl)

/-! ## VII · the triaxial seal -/

/-- ACTUATION IS TRIAXIAL: every actuation carries a fold that is an
involution, a registration that
keeps the height, forgets the side and is the only such map onto the
seat, and a seat where both fix
the act and off which neither does; off the seat the record leaves
exactly the orientation; every act
and its reversal instance the root; and the axes force no
orientation. -/
theorem actuation_is_triaxial (A : Actuation R) :
(∀ p, fold A (fold A p) = p) ∧
(∀ p, reg A (fold A p) = reg A p ∧ (reg A p).2 = p.2 ∧ fold A
(reg A p) = reg A p) ∧
(∀ g : A.dir.Side × A.Height → A.dir.Side × A.Height,
(∀ p, (g p).2 = p.2) → (∀ p, OnSeat A (g p)) → ∀ p, g p = reg
A p) ∧
(∀ p, OnSeat A p → fold A p = p ∧ reg A p = p) ∧
(∀ p, ¬ OnSeat A p → fold A p ≠ p ∧ reg A p ≠ p) ∧
(∀ p, ¬ OnSeat A p → R ∧ R ∧ fold A p ≠ p ∧ reg A (fold A p) =
reg A p) :=
(fold_involutive A, fun p => (reg_forgets_side A p,
reg_keeps_height A p, fold_of_reg A p),
registration_is_forced A, seat_cancels_both A, off_seat_both A,
axes_force_no_orientation A)

/-- THE TRIAXIAL SEAL: actuation is triaxial, universally; the
bridge atom is an actuation; and it is
not characterless. -/
theorem the_triaxial_seal :
(∀ (R : Prop) (A : Actuation R), (∀ p, fold A (fold A p) = p) ∧
(∀ p, OnSeat A p → fold A p = p ∧ reg A p = p) ∧
(∀ p, ¬ OnSeat A p → fold A p ≠ p ∧ reg A (fold A p) = reg A
p)) ∧
(∀ d t : Int, OnSeat atom (d, t) ↔ d = 0) ∧
(fold atom ((1 : Int), (0 : Int)) ≠ ((1 : Int), (0 : Int)) ∧ reg
atom (fold atom ((1 : Int), (0 : Int))) = reg atom ((1 : Int), (0
: Int))) :=
(fun _ A => (fold_involutive A, seat_cancels_both A,
fun p h => record_leaves_the_orientation A p h),
fun d t => seat_iff_zero_side atom (d, t),
the_atom_is_not_characterless)

/-! ## VIII · the Tongue: freedom and the only special cut -/

/-- TONGUE FREEDOM: no reading of the record returns the
orientation. If a record is blind to the fold
and an orientation is flipped by it, the orientation is not a
function of the record. -/
theorem tongue_freedom {α β : Type} (τ : α → α) (ρ : α → β) (g : β →
Bool) (d : α → Bool) (x : α)
(blind : ρ (τ x) = ρ x) (flip : d (τ x) = !d x) : d ≠ fun y => g
(p y) := by
intro h
have h1 : d (τ x) = g (ρ (τ x)) := congrFun h (τ x)
have h2 : d x = g (ρ x) := congrFun h x
rw [blind] at h1
rw [h2] at h1
rw [h1] at flip
cases hd : d x with
| false => rw [hd] at flip; exact Bool.noConfusion flip
| true => rw [hd] at flip; exact Bool.noConfusion flip

/-- THE ONLY SPECIAL CUT: the seat is exactly where both blind spots
cancel; nowhere else do the fold
and the registration both fix an act. -/

```

```

theorem the_only_special_cut (A : Actuation R) (p : A.dir.Side ×
A.Height) :
(fold A p = p ∧ reg A p = p) ↔ OnSeat A p :=
(fun h => h.1, fun h => seat_cancels_both A p h)

/-! ## IX · every root actuates: the root carries the three axes -/

/-- THE REGISTRATION IS THE FOLD'S RECORD: it reads an act and its
reversal alike and keeps the height, and two
acts have one record exactly when they have one height. -/
theorem registration_is_the_folds_record (A : Actuation R) (p q :
A.dir.Side × A.Height) :
reg A (fold A p) = reg A p ∧ (reg A p = reg A q ↔ p.2 = q.2) :=
(rfl, (fun h => show (reg A p).2 = (reg A q).2 from congrArg
Prod.snd h, fun h => by
cases p with
| mk s t =>
cases q with
| mk s' t' =>
have ht : t = t' := h
show (A.dir.zero, t) = (A.dir.zero, t')
rw [ht]))

/-- THE REGISTRATION LANDS ON THE SEAT: every record is a seat
point. -/
theorem reg_lands_on_seat (A : Actuation R) (p : A.dir.Side ×
A.Height) : OnSeat A (reg A p) :=
fold_of_reg A p

/-- THE ORIENTATION IS OPEN AT THE SEAT: off the seat, an act and
its reversal land on one seat point, the record
of both; at that place the record has left exactly the orientation
open. -/
theorem open_at_the_seat (A : Actuation R) (p : A.dir.Side ×
A.Height) (h : ¬ OnSeat A p) :
OnSeat A (reg A p) ∧ reg A (fold A p) = reg A p ∧ fold A p ≠ p :=
(reg_lands_on_seat A p, rfl, h)

/-- TWO AXES LEAVE TWO: off the seat, every reading blind to the
fold, the record among them, reads an act and
its reversal alike: two acts, one reading. -/
theorem even_pair_leaves_two (A : Actuation R) (p : A.dir.Side ×
A.Height) (h : ¬ OnSeat A p) {α : Type}
(f : A.dir.Side × A.Height → α) (blind : ∀ q, f (fold A q) = f
q) :
fold A p ≠ p ∧ f (fold A p) = f p ∧ reg A (fold A p) = reg A p :=
(h, blind p, rfl)

/-- THREE AXES LOCK ONE: the record together with the orientation,
the side, determines the act. -/
theorem orientation_locks_the_act (A : Actuation R) (p q :
A.dir.Side × A.Height)
(hr : reg A p = reg A q) (hs : p.1 = q.1) : p = q := by
cases p with
| mk s t =>
cases q with
| mk s' t' =>
have ht : t = t' := congrArg Prod.snd hr
have hs' : s = s' := hs
rw [hs', ht]

/-- A degenerate direction has the zero side and no other. -/
def Direction.Degenerate (D : Direction) : Prop := ∀ s, s = D.zero

/-- THE CHARACTERLESS CASE: over a degenerate direction every act is
on the seat, and the fold and the
registration are both the identity: one scalar, no fold, no seat to
lock. -/
theorem degenerate_is_characterless (A : Actuation R) (h :
A.dir.Degenerate) (p : A.dir.Side × A.Height) :
OnSeat A p ∧ fold A p = p ∧ reg A p = p :=
have hs : OnSeat A p := (seat_iff_zero_side A p).mpr (h p.1)
(hs, seat_cancels_both A p hs)

/-- The integer direction is not degenerate. -/
theorem intDir_not_degenerate : ¬ intDir.Degenerate := fun h =>
have h0 : (1 : Int) = 0 := h (1 : Int)
absurd h0 (by decide)

/-- EVERY ROOT ACTUATES: a self-grounding root is the root of the
actuation over its own acts, with the integer
direction; the root's deeds sit at the zero side and instance the
root as before. -/
def SelfGrounding.toActuation (S : SelfGrounding R) : Actuation R :=

```

```

(intDir, S.Act, ((0 : Int), S.anAct), fun p => S.instances p.2)

theorem every_root_actuates (S : SelfGrounding R) :
  (∀ a : S.Act, OnSeat S.toActuation ((0 : Int), a)) ∧
  (∀ a : S.Act, ¬ OnSeat S.toActuation ((1 : Int), a)) ∧
  ¬ S.toActuation.dir.Degenerate :=
  (fun _ => rfl,
   fun a h =>
     have h0 : (1 : Int) = 0 := (seat_iff_zero_side S.toActuation
      ((1 : Int), a)).mp h
     absurd h0 (by decide),
    intDir_not_degenerate)

/-- THE ROOT CARRIES THREE AXES: for every self-grounding root, its
actuation carries the fold, an involution; the
registration, keeping the height, forgetting the side, the only
height-keeping map onto the seat; and the seat,
where both fix the act; and it is not characterless: at every height
an act off the seat, its reversal a second
act with the same record, both instantiating the root, the orientation
between them forced by nothing. -/
theorem the_root_carries_three_axes (S : SelfGrounding R) :
  (∀ p, fold S.toActuation (fold S.toActuation p) = p) ∧
  (∀ p, reg S.toActuation (fold S.toActuation p) = reg
   S.toActuation p ∧ (reg S.toActuation p).2 = p.2) ∧
  (∀ g : Int × S.Act → Int × S.Act, (∀ p, (g p).2 = p.2) → (∀ p,
   OnSeat S.toActuation (g p) →
    ∀ p, g p = reg S.toActuation p) ∧
   (∀ p, OnSeat S.toActuation p → fold S.toActuation p = p ∧ reg
    S.toActuation p = p) ∧
   (∀ a : S.Act, fold S.toActuation ((1 : Int), a) ≠ ((1 : Int), a) ∧
    reg S.toActuation (fold S.toActuation ((1 : Int), a)) = reg
    S.toActuation ((1 : Int), a)) ∧
   (∀ p, ¬ OnSeat S.toActuation p →
    R ∧ R ∧ fold S.toActuation p ≠ p ∧ reg S.toActuation (fold
    S.toActuation p) = reg S.toActuation p) :=
  (fold_involutive S.toActuation,
   fun p => (reg_forgets_side S.toActuation p, reg_keeps_height
    S.toActuation p),
   registration_is_forced S.toActuation,
   seat_cancels_both S.toActuation,
   fun a => record_leaves_the_orientation S.toActuation ((1 : Int),
    a) ((every_root_actuates S).2.1 a),
   axes_force_no_orientation S.toActuation)

end TriAct

/-! ## Cones, pinned as printed: every theorem on no axiom -/
/-- info: 'TriAct.fold_involutive' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.fold_involutive
/-- info: 'TriAct.reg_forgets_side' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_forgets_side
/-- info: 'TriAct.reg_keeps_height' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_keeps_height
/-- info: 'TriAct.fold_of_reg' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.fold_of_reg
/-- info: 'TriAct.reg_idempotent' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.reg_idempotent
/-- info: 'TriAct.registration_is_forced' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.registration_is_forced
/-- info: 'TriAct.seat_iff_zero_side' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.seat_iff_zero_side
/-- info: 'TriAct.seat_cancels_both' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.seat_cancels_both
/-- info: 'TriAct.off_seat_both' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.off_seat_both
/-- info: 'TriAct.record_leaves_the_orientation' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.record_leaves_the_orientation
/-- info: 'TriAct.even_readings_are_blind' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.even_readings_are_blind
/-- info: 'TriAct.readings_of_the_record_are_blind' does not depend
on any axioms -/
#guard_msgs in #print axioms TriAct.readings_of_the_record_are_blind
/-- info: 'TriAct.denial_instances_root' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.denial_instances_root
/-- info: 'TriAct.root_holds' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.root_holds

/-- info: 'TriAct.axes_force_no_orientation' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.axes_force_no_orientation
/-- info: 'TriAct.int_neg_neg' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.int_neg_neg
/-- info: 'TriAct.self_neg_zero' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.self_neg_zero
/-- info: 'TriAct.the_bridge_atom' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.the_bridge_atom
/-- info: 'TriAct.the_atom_is_not_characterless' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.the_atom_is_not_characterless
/-- info: 'TriAct.actuation_is_tri axial' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.actuation_is_tri axial
/-- info: 'TriAct.the_tri axial_seal' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.the_tri axial_seal
/-- info: 'TriAct.tongue_freedom' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.tongue_freedom
/-- info: 'TriAct.the_only_special_cut' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.the_only_special_cut
/-- info: 'TriAct.registration_is_the_folds_record' does not depend
on any axioms -/
#guard_msgs in #print axioms TriAct.registration_is_the_folds_record
/-- info: 'TriAct.reg_lands_on_seat' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.reg_lands_on_seat
/-- info: 'TriAct.open_at_the_seat' does not depend on any axioms -/
#guard_msgs in #print axioms TriAct.open_at_the_seat
/-- info: 'TriAct.even_pair_leaves_two' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.even_pair_leaves_two
/-- info: 'TriAct.orientation_locks_the_act' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.orientation_locks_the_act
/-- info: 'TriAct.degenerate_is_characterless' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.degenerate_is_characterless
/-- info: 'TriAct.intDir_not_degenerate' does not depend on any
axioms -/
#guard_msgs in #print axioms TriAct.intDir_not_degenerate
/-- info: 'TriAct.every_root_actuates' does not depend on any axioms
-/
#guard_msgs in #print axioms TriAct.every_root_actuates
/-- info: 'TriAct.the_root_carries_three_axes' does not depend on
any axioms -/
#guard_msgs in #print axioms TriAct.the_root_carries_three_axes

/-
Root_Grade_Ledger.lean · the root at theorem grade, and RA-Tongue
co-location with it

Core Lean 4: no import, no axiom declared, no sorry, no
native_decide, and every theorem rests on no
axiom at all, not even propositional extensionality, quotients or
choice.

The floor Codex (Master Codex v5.1.0, Codex.lean) grades every
claim on one ledger and joins claims
at the weakest link. It declares the root as `axiom RA` and pins
`raClaim.grade = .premise`; its
co-location claim, the root joined with the wall, therefore
inherits `.premise`. This kernel carries
the same ledger and the same join law with no axiom declared,
warrants the root by a theorem with no
axiom and no hypothesis, and proves that the co-location claim
then stands at theorem grade.
I The grade ledger and the weakest-link join, as in the Codex.
II The root, a theorem: the constructed domain of TOE_Zero.lean,
warranted on no axiom.
III The wall, a theorem: the Codex's K4 frame, its target not a
function of its record, proved on
no axiom through Tongue freedom.
IV The claims: the root at theorem grade, the wall at theorem
grade, and their co-location.
V Why the Codex read premise: a join's grade is its weakest
link, and co-location's grade is the
root's grade exactly; with the root at theorem grade,
co-location is a theorem.
VI The ledger seal.
VII The grade read from the warrant, as the Codex reads it: a
root with a warrant, a proof of its

```

```

    statement, is theorem grade; a root posited without one, the
    Codex's `axiom RA`, is premise grade;
    the two claims of section IV carry exactly the grades this
    rule reads.
-/

namespace RGL

/-! ## I · the grade ledger, as in the Codex -/

inductive Grade : Type
| premise | corroboration | operational | structural | engineering
| conditional | theoremConditional | analytic | theorem
deriving DecidableEq

def Grade.rank : Grade → Nat
| .premise => 0 | .corroboration => 1 | .operational => 2 |
|.structural => 3
| .engineering => 4 | .conditional => 5 | .theoremConditional => 6
| .analytic => 7 | .theorem => 8

def Grade.weakest (a b : Grade) : Grade := if a.rank ≤ b.rank then a
else b

/-- A claim carries its statement, its grade and its warrant, a
proof of the statement. -/
structure Claim : Type where
  stmt    : Prop
  grade   : Grade
  warrant : stmt

/-- Two claims join at their weakest link. -/
def Claim.join (c₁ c₂ : Claim) : Claim where
  stmt    := c₁.stmt ∧ c₂.stmt
  grade   := Grade.weakest c₁.grade c₂.grade
  warrant := (c₁.warrant, c₂.warrant)

theorem join_grade (c₁ c₂ : Claim) : (Claim.join c₁ c₂).grade =
  Grade.weakest c₁.grade c₂.grade := rfl

/-! ## II · the root, a theorem (the constructed domain of
TOE_Zero.lean) -/

structure SelfGrounding (R : Prop) where
  Act      : Type
  anAct    : Act
  instances : Act → R

theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R :=
  G.instances G.anAct

def ΔE₀ : Unit → Int := fun _ => 1
def RA₀ : Prop := ∀ x : Unit, 0 < ΔE₀ x
def ra₀ : SelfGrounding RA₀ := (Unit, (), fun _ => show (0 : Int)
< 1 by decide)

/-- THE ROOT ON THE CONSTRUCTED DOMAIN, A THEOREM: no axiom, no
hypothesis. -/
theorem root_on_the_constructed_domain : RA₀ := seated_undeniable
ra₀

/-! ## III · the wall, a theorem (the Codex's K4 frame) -/

/-- Tongue freedom: if a record is blind to the fold and an
orientation is flipped by it, the
orientation is not a function of the record. -/
theorem tongue_freedom {α β : Type} (τ : α → α) (ρ : α → β) (g : β →
Bool) (d : α → Bool) (x : α)
  (blind : ρ (τ x) = ρ x) (flip : d (τ x) = !d x) : d ≠ fun y => g
(ρ y) := by
  intro h
  have h1 : d (τ x) = g (ρ (τ x)) := congrFun h (τ x)
  have h2 : d x = g (ρ x) := congrFun h x
  rw [blind] at h1
  rw [← h2] at h1
  rw [h1] at flip
  cases hd : d x with
  | false => rw [hd] at flip; exact Bool.noConfusion flip
  | true => rw [hd] at flip; exact Bool.noConfusion flip

namespace K4

def val : Nat → Nat
| 0 => 11 | 1 => 851 | 2 => 13 | 3 => 1273 | 4 => 17 | 5 => 437

```

```

| 6 => 19 | 7 => 2119 | 8 => 23 | 9 => 1703 | 10 => 29 | 11 => 869
| _ => 0

def tau : Nat → Nat := fun i =>
  if i < 12 then (if i % 2 = 0 then i + 1 else i - 1) else i

def target : Nat → Bool := fun i => i % 2 == 0

def rho : Nat → Nat := fun i =>
  (val i % 2) * 1000 + (val i % 3) * 100 + (val i % 5) * 10 + (val i
% 7)

/-- The record is blind to the fold at the first pair. -/
theorem register_even_at_zero : rho (tau 0) = rho 0 := by decide

/-- The target is flipped by the fold at the first pair. -/
theorem target_odd_at_zero : target (tau 0) = !target 0 := by decide

/-- THE WALL: no reading of the record returns the target. -/
theorem wall (g : Nat → Bool) : target ≠ fun y => g (rho y) :=
  tongue_freedom tau rho g target 0 register_even_at_zero
  target_odd_at_zero

end K4

/-! ## IV · the claims -/

/-- The root at theorem grade, warranted by a theorem with no axiom
and no hypothesis. -/
def raClaim₀ : Claim where
  stmt    := RA₀
  grade   := .theorem
  warrant := root_on_the_constructed_domain

/-- The wall at theorem grade, as in the Codex. -/
def wallClaim : Claim where
  stmt    := ∀ g : Nat → Bool, K4.target ≠ fun y => g (K4.rho y)
  grade   := .theorem
  warrant := K4.wall

/-- RA-Tongue co-location: the root joined with the wall, as in the
Codex. -/
def colocClaim₀ : Claim := Claim.join raClaim₀ wallClaim

/-- The Codex's root, graded as the Codex grades it, for comparison:
the same statement form, at
premise grade. -/
def raClaimPremise : Claim where
  stmt    := RA₀
  grade   := .premise
  warrant := root_on_the_constructed_domain

/-! ## V · why the Codex read premise, and what co-location is now -/

/-- A join with a theorem-grade claim has exactly the other claim's
grade. -/
theorem weakest_with_theorem (g : Grade) : Grade.weakest g .theorem
= g := by
  cases g <|> rfl

/-- CO-LOCATION'S GRADE IS THE ROOT'S GRADE, EXACTLY: the wall is a
theorem, so the join reads the
root's grade and nothing else. -/
theorem coloc_grade_is_the_roots_grade (r : Claim) :
  (Claim.join r wallClaim).grade = r.grade := by
  show Grade.weakest r.grade Grade.theorem = r.grade
  exact weakest_with_theorem r.grade

/-- WHY THE CODEX READ PREMISE: with the root labelled premise, the
co-location claim is premise. -/
theorem premise_root_gives_premise_coloc : (Claim.join
raClaimPremise wallClaim).grade = .premise := rfl

/-- THE ROOT IS A THEOREM-GRADE CLAIM. -/
theorem ra_is_theorem_grade : raClaim₀.grade = .theorem := rfl

/-- RA-TONGUE CO-LOCATION IS A THEOREM-GRADE CLAIM. -/
theorem coloc_is_theorem_grade : colocClaim₀.grade = .theorem := rfl

/-- The co-location claim's statement holds: the root and the wall,
both proved. -/
theorem coloc_holds : colocClaim₀.stmt := colocClaim₀.warrant

/-! ## VI · the ledger seal -/

```

```

/-- THE LEDGER SEAL: the root is a theorem on no axiom and no
hypothesis; the wall is a theorem; their
co-location holds and stands at theorem grade; and co-location's
grade is the root's grade exactly,
which is why the Codex, labelling the root premise, read premise. -/
theorem the_ledger_seal :
  RA0 ∧ (∀ g : Nat → Bool, K4.target ≠ fun y => g (K4.rho y)) ∧
  colocClaim0.stmt ∧ colocClaim0.grade = .theorem ∧
  (∀ r : Claim, (Claim.join r wallClaim).grade = r.grade) ∧
  (Claim.join raClaimPremise wallClaim).grade = .premise :=
  (root_on_the_constructed_domain, K4.wall, coloc_holds,
  coloc_is_theorem_grade,
  coloc_grade_is_the_roots_grade, premise_root_gives_premise_coloc)

/-! ## VII · the grade read from the warrant, as the Codex reads it
-/

/-- The evidence a ledger entry carries for a statement: posited
with no warrant, or warranted by a proof. -/
inductive Evidence (P : Prop) : Type
| posited
| warranted (w : P)

/-- THE CODEX'S RULE FOR THE ROOT'S GRADE: evidence at theorem grade
is a proof with its axioms printed. A root
that comes with a warrant, a proof of its statement, is theorem
grade; a root posited with none, as the Codex
declares `axiom RA`, is premise grade. -/
def rootGrade : Evidence RA0 → Grade
| .warranted _ => .theorem
| .posited      => .premise

/-- A WARRANTED ROOT IS THEOREM GRADE: the root of section II
carries its warrant, so the rule reads theorem. -/
theorem a_warranted_root_is_theorem_grade :
  rootGrade (.warranted root_on_the_constructed_domain) = .theorem
:= rfl

/-- AN UNWARRANTED ROOT IS PREMISE GRADE: with no warrant the rule
reads premise, the Codex's own reading of
its declared axiom. -/
theorem an_unwarranted_root_is_premise_grade : rootGrade .posited =
.premise := rfl

/-- THE LEDGER READS THE WARRANT: the root claim of section IV
carries the grade the rule reads from its warrant,
and the comparison claim carries the grade the rule reads from none.
-/
theorem the_ledger_reads_the_warrant :
  raClaim0.grade = rootGrade (.warranted raClaim0.warrant) ∧
  raClaimPremise.grade = rootGrade .posited :=
  (rfl, rfl)

/-- THE GRADE IS NOT ELECTED: the rule reads theorem from a warrant
and premise from none, never the other way;
whatever label an entry carries, the warranted root's grade under
the rule is theorem. -/
theorem the_grade_is_not_elected :
  (∀ w : RA0, rootGrade (.warranted w) = .theorem) ∧ rootGrade
.posited ≠ .theorem ∧
  (∀ w : RA0, rootGrade (.warranted w) ≠ rootGrade .posited) :=
  (fun _ => rfl, fun h => Grade.noConfusion h, fun _ h =>
  Grade.noConfusion h)

end RGL

/-! ## Cones, pinned as printed: every theorem on no axiom -/
/-- info: 'RGL.join_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.join_grade
/-- info: 'RGL.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.seated_undeniable
/-- info: 'RGL.root_on_the_constructed_domain' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.root_on_the_constructed_domain
/-- info: 'RGL.tongue_freedom' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.tongue_freedom
/-- info: 'RGL.K4.register_even_at_zero' does not depend on any
axioms -/
#guard_msgs in #print axioms RGL.K4.register_even_at_zero
/-- info: 'RGL.K4.target_odd_at_zero' does not depend on any axioms
-/
#guard_msgs in #print axioms RGL.K4.target_odd_at_zero
/-- info: 'RGL.K4.wall' does not depend on any axioms -/

```

```

#guard_msgs in #print axioms RGL.K4.wall
/-- info: 'RGL.weakest_with_theorem' does not depend on any axioms
-/
#guard_msgs in #print axioms RGL.weakest_with_theorem
/-- info: 'RGL.coloc_grade_is_the_roots_grade' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.coloc_grade_is_the_roots_grade
/-- info: 'RGL.premise_root_gives_premise_coloc' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.premise_root_gives_premise_coloc
/-- info: 'RGL.ra_is_theorem_grade' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.ra_is_theorem_grade
/-- info: 'RGL.coloc_is_theorem_grade' does not depend on any axioms
-/
#guard_msgs in #print axioms RGL.coloc_is_theorem_grade
/-- info: 'RGL.coloc_holds' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.coloc_holds
/-- info: 'RGL.the_ledger_seal' does not depend on any axioms -/
#guard_msgs in #print axioms RGL.the_ledger_seal
/-- info: 'RGL.a_warranted_root_is_theorem_grade' does not depend on
any axioms -/
#guard_msgs in #print axioms RGL.a_warranted_root_is_theorem_grade
/-- info: 'RGL.an_unwarranted_root_is_premise_grade' does not depend
on any axioms -/
#guard_msgs in #print axioms
  RGL.an_unwarranted_root_is_premise_grade
/-- info: 'RGL.the_ledger_reads_the_warrant' does not depend on any
axioms -/
#guard_msgs in #print axioms RGL.the_ledger_reads_the_warrant
/-- info: 'RGL.the_grade_is_not_elected' does not depend on any
axioms -/
#guard_msgs in #print axioms RGL.the_grade_is_not_elected

```

AUTHOR'S PROVENANCE AND METHOD DISCLOSURE

This paper was developed under Trisduction, a verification and organizing discipline that adds the cited results no warrant; the method and its executable batteries are stated in full in the master reference (Islam 2026j).

Author and method. Mohammad F. Islam, PhD, independent researcher, sole author and sole authority for every claim and every error. The reduction was directed by the author and drafted with Claude (Anthropic) as scribe: the model wrote and compiled the kernel, checked citations and argued against the author's positions on instruction; it holds no authorship. One claim was held at the model's insistence: the kernel reads the Perelman functional by its shape and does not formalize it, and the paper says so wherever the witness is named: the Perelman witness, in the shape the kernel types, strict descent off the seat, supplies the act and is a strict subtype of the bridge, and the historical functional inhabits that shape on the surgical route where its descent is strict, as Section 10 states.

The register. In the programme's vocabulary: the form, the act's identity with the value, the universal closure, the bridge and the typing of the witness stand at $[\{\epsilon\}, T]$ on no axiom; the triaxial reading at $[\{\epsilon\}, S]$; the value in every dimension of the generalized row at the grade of the cited theorems; $\Delta M = 0$ on the cited topology. The act is the row's least-erasure posit read as existence at rest; the seat is the round state. The Grand Witness of the author's Offering Bit is placed by theorem: an offering of the act, one supply among those the act admits. The freedom cut, prime-as-freedom and the triaxial lock are applied, with the lessons of the six earlier closures combined.

Audit. This edition has not yet been prosecuted by the external substrates, and no audit result is claimed. The development provenance is the author's Zenodo trail, the Unabridged Trisduction Codex and the Lean Codex at github.com/1000sapients/Trisduction.

Nature of Root Axiom

The root is triaxial, and that is the force of the cut. A characterless existence has no triaxiality: one scalar, no fold, no seat, nothing to lock. The root, to exist is to actuate, carries three axes, and they are not chosen: for every actuation, the direction of its deed gives the fold, the registration and the seat, and the registration is the only height-keeping map onto the seat. The deed is the direction: a denial spends the deed and instances the root. The registration is the fold's record: it keeps the height and forgets the side, and the two even axes, symmetry and record, are closed under combination and blind. The seat is the fixed set, where the two blind spots cancel, and the third axis is the one orientation the even pair cannot supply. Two axes never determine the point; three do. The seven rows are one cut because each is an involution with a seat and one missing orientation. The root makes the third axis native as a place: the seat, and the one orientation still open there; the axes themselves force no orientation. Which way that orientation points is the value, and it is the root read on that seat. The root alone still forces no value.

Key to the statement, in the words of its receipts. The three axes are the two even axes, the fold's symmetry and the record, and the orientation, the side of a point of the actuation, which the even pair cannot supply; the fold, the registration and the seat are what the direction of the deed gives, and the seat is where the side is read. The point two axes never determine is a point off the seat: there the even pair does not tell a point from its reversal, and the record with the side locks the point; on the seat both blind spots cancel and nothing is open there to determine. A point of the actuation is a side and a height, and every point instances the root: the points are the actuation's deeds, and the deed is the direction in the sense that its side is its direction, reversed by the fold. The characterless case, one scalar, is a degenerate direction: the fold is the identity and the seat is total, which is to say no fold and no seat to lock. The axes are not chosen in the sense that, given the direction, the fold, the registration and the seat are forced, the registration the only height-keeping map onto the seat; the root's own actuation takes the integer direction, under which it is not characterless, while a degenerate direction over the same deeds would be (`every_root_actuates`, `degenerate_is_characterless`). The root read on that seat is the root read on the row at the seat point where a point and its reversal land with one record; that the orientation between them is the row's value bit is the paper's structural reading, a second statement beside its kernel's theorems, and no kernel states a map between the two. Each row carries the actuation's seat as the root of an actuation over its own deeds; a row's own seat, where its kernel states one, is a further statement of that kernel, named in the last row of the table, and on P versus NP it is vacant. A deed of the root is an inhabitant of its type of acts, which the root kernel's own comments call an act; in the root's actuation those deeds sit at the zero side, on the seat, and the points off the seat are the sides the direction supplies; the act, unqualified, is existence read on the row, the field the closing theorem consumes.

Receipts, each cited for what its statement says; R.1, R.2 and R.3 are the three kernels of Appendix R, every theorem of each on no axiom; the three theorems of the Master Codex are cited, not carried.

Clause of the statement	Receipt
the root carries three axes, not chosen: every self-grounding root is the root of an actuation over its own deeds, and for every actuation, given the direction of its deed, the fold, the registration and the seat are forced, the registration the only height-keeping map onto the seat	<code>every_root_actuates</code> , <code>the_root_carries_three_axes</code> , <code>actuation_is_triaxial</code> , <code>registration_is_forced</code> (Appendix R.2)
the registration is the only height-keeping map onto the seat	<code>registration_is_forced</code> (R.2)
the axes force no orientation	<code>axes_force_no_orientation</code> (R.2)
the deed is the direction: a deed of the actuation is a point, a side and a height, its side its direction, reversed by the fold; a reversed point is still a deed and instances the root; a denial spends the deed and instances the root	<code>fold</code> , <code>denial_instances_root</code> (R.2); <code>denial_reenacts_root</code> , <code>denial_instantiates</code> (R.1)
the registration is the fold's record: it keeps the height, forgets the side and reads a point and its reversal alike; the fold is an involution; the even pair is closed under combination	<code>registration_is_the_folds_record</code> , <code>reg_forgets_side</code> , <code>reg_keeps_height</code> , <code>fold_involution</code> , <code>fold_of_reg</code> (R.2)
the two even axes are blind: no reading of the record returns the orientation; symmetry keyless, the line property keyed	<code>readings_of_the_record_are_blind</code> , <code>tongue_freedom</code> (R.2); <code>symmetry_is_keyless</code> , <code>line_property_is_keyed</code> , <code>exactly_one_stands</code> (Islam 2026l, Master Codex v5.1.0, commit 5e55fca, cited for their statements, not carried; that Codex declares its root as an axiom, this paper's kernels declare none)
the seat is the fixed set, where both blind spots cancel; off the seat both stand	<code>seat_iff_zero_side</code> , <code>seat_cancels_both</code> , <code>off_seat_both</code> , <code>the_only_special_cut</code> (R.2)
the registration lands on the seat; off the seat a point and its reversal land on one seat point with one record, and the orientation is what stays open there; every reading of the record reads both alike	<code>reg_lands_on_seat</code> , <code>open_at_the_seat</code> , <code>record_leaves_the_orientation</code> , <code>readings_of_the_record_are_blind</code> (R.2)
the characterless case, no fold and no seat to lock: over a degenerate direction, one side only, the fold is the identity and every point is on the seat; the root's actuation is not degenerate	<code>degenerate_is_characterless</code> , <code>intDir_not_degenerate</code> , <code>every_root_actuates</code> (R.2)

Clause of the statement	Receipt
the bridge atom is an actuation, and not characterless	<code>the_bridge_atom, the_atom_is_not_characterless, the_tri axial_seal (R.2)</code>
the scalar ground in the algebra, a third reading beside the actuation's: the Return, the line the conjugation fixes and the twelve gates, bound with the root	<code>the_return, the_line_is_the_fixed_set, twenty_four_units_twelve_gates, the_class_equation, the_master_seal (R.1)</code>
the root at theorem grade on the Codex's ledger rule, the grade read from the evidence the root is entered with, a warrant reading theorem and a posit premise, and not elected, and RA--Tongue co-location inheriting it	<code>a_warranted_root_is_theorem_grade, the_ledger_reads_the_warrant, the_grade_is_not_elected, ra_is_theorem_grade, coloc_grade_is_the_roots_grade, the_ledger_seal (R.3)</code>
two axes never determine the point; three do: off the seat the even pair does not tell a point from its reversal, and the record with the side, which carries the orientation, locks the point; on the seat both blind spots cancel and nothing is open to determine	<code>even_pair_leaves_two, orientation_locks_the_act, seat_cancels_both (R.2); in its own coordinates, for independent axes in $\mathbb{F}_2^3$, two_axes_leave_two, three_axes_lock_one (this paper's kernel), a second statement</code>
the root alone forces no value; the root read on the row is the value	<code>undeniable_root_forces_no_value; root_on_row_is_the_value, act_is_the_value (this paper's kernel)</code>
the seven rows are one cut: each row's root is an actuation, so each carries an involution, the fold, with a seat and one missing orientation	<code>every_root_actuates, the_root_carries_three_axes (R.2), stated for every self-grounding root and read at the constructed root ra_s of this paper's kernel, whose structure repeats R.2's SelfGrounding field for field; the row's own seat is the round point, fixed under the step (<code>seat_is_fixed</code>), a statement of this kernel beside the actuation's seat; the kernel states no involution of its own beyond the actuation's fold; the row ledger of Islam (2026f, Book III), cited</code>

REFERENCES

- Cao, H.-D. and X.-P. Zhu. 2006. A complete proof of the Poincaré and geometrization conjectures: application of the Hamilton--Perelman theory of the Ricci flow. *Asian Journal of Mathematics* 10: 165--492.
- de Moura, L. and S. Ullrich. 2021. The Lean 4 theorem prover and programming language. *Automated Deduction, CADE 28*, 625--635.
- Freedman, M. H. 1982. The topology of four-dimensional manifolds. *Journal of Differential Geometry* 17: 357--453.
- Hamilton, R. S. 1982. Three-manifolds with positive Ricci curvature. *Journal of Differential Geometry* 17: 255--306.
- Islam, M. F. 2026a. A formal completed proof of the Navier--Stokes closure from existence alone. Zenodo. doi:10.5281/zenodo.22670344.
- Islam, M. F. 2026b. What exists is realized: the completed formal closure of the Hodge question. Zenodo.

doi:10.5281/zenodo.22701510.

- Islam, M. F. 2026c. The cut-agnostic division theorem: why every open problem is exactly its proved part and its unicorn. Zenodo. doi:10.5281/zenodo.22954862.
- Islam, M. F. 2026d. The veil and the wall. Zenodo. doi:10.5281/zenodo.21389758.
- Islam, M. F. 2026e. A formal proof of the Riemann Hypothesis by least erasure. Zenodo. doi:10.5281/zenodo.23034066.
- Islam, M. F. 2026f. Nothing escapes least erasure. Zenodo. doi:10.5281/zenodo.23080221.
- Islam, M. F. 2026g. PhysOST^T: the Trisduction physical operating system. Zenodo. doi:10.5281/zenodo.23117440.
- Islam, M. F. 2026h. The offering bit: one theory, one hypothesis, one witness offering. Zenodo. doi:10.5281/zenodo.22735236.
- Islam, M. F. 2026i. Theory of Theories of Everything (TOE of All TOEs): existence proves existence only by motion. Zenodo. doi:10.5281/zenodo.23168379.
- Islam, M. F. 2026j. TRISDUCTION: A linguistically, topologically, and mathematically sealed verification architecture. Triaxial orthogonality, twelve-gate closure, the quaternionic completion, the root axiom, and the master pre-sealed proposition ledger. Zenodo, version 4, 19 June 2026. doi:10.5281/zenodo.20757507. Mirror: PhilArchive record ISLTG. Master reference, continuously updated at the same location.
- Islam, M. F. 2026k. Seven rows, one root: the master plan of the series, revision F.5. Manuscript of record, 5 October 2026, carried with the series; its Appendix R kernels are carried into this paper's Appendix R, each extended by one additive section.
- Islam, M. F. 2026l. Trisduction: the Master Codex, edition 5.1.0. Codex.lean and its kernels, commit 5e55fca98c2f, master SHA-256 441a485d7c3bd3ce. Repository 1000sapients/Trisduction, GitHub.
- Kleiner, B. and J. Lott. 2008. Notes on Perelman's papers. *Geometry & Topology* 12: 2587--2855.
- Milnor, J. 2006. The Poincaré conjecture. In *The Millennium Prize Problems*, 71--83. Clay Mathematics Institute.
- Morgan, J. and G. Tian. 2007. *Ricci Flow and the Poincaré Conjecture*. Clay Mathematics Monographs 3. American Mathematical Society.
- Newman, M. H. A. 1966. The engulfing theorem for topological manifolds. *Annals of Mathematics* 84: 555--571.
- Perelman, G. 2002. The entropy formula for the Ricci flow and its geometric applications. arXiv:math/0211159.
- Perelman, G. 2003. Ricci flow with surgery on three-manifolds. arXiv:math/0303109. Finite extinction time for the solutions to the Ricci flow on certain three-manifolds. arXiv:math/0307245.
- Poincaré, H. 1904. Cinquième complément à l'Analysis situs. *Rendiconti del Circolo Matematico di Palermo* 18: 45--110.
- Smale, S. 1961. Generalized Poincaré's conjecture in dimensions greater than four. *Annals of Mathematics* 74: 391--406.

End of manuscript